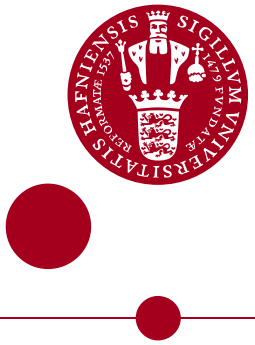
UNIVERSITY OF COPENHAGEN
FACULTY OF SOCIAL SCIENCES



Master Thesis in Economics

Buffer-Stock Households as Reinforcement Learners

Andreas Marius Laursen

Supervisor: Thomas Høgholm Jørgensen

March 4, 2025

Abstract

I study reinforcement learning (RL) households in the Buffer-Stock model. While economic agents are typically assumed to solve their problem perfectly, the RL household must learn a consumption-saving policy using the experience it gathers from simulating itself in the model. Using the Deep Deterministic Policy Gradient algorithm, I train an RL household in the finite-horizon Buffer-Stock model and then test it in life-cycle simulations, where its behavior is compared to the optimal dynamic programming solution. The RL household tends to act rationally and does achieve close to optimal behavior in some cases. However, in other cases, it deviates more distinctly from optimal behavior. While these deviations may be seen as a result of the RL household's bounded rationality, its behavior remains opaque and difficult to interpret from an economic perspective.

Table of Contents

1 Introduction

In economic models, agents are typically assumed to solve their optimization problem perfectly. In this thesis, I view economic agents as imperfect problem solvers, where they must learn their policy by simulating themselves in the model. The framework for modeling this is *reinforcement learning*.

Reinforcement learning is a type of machine learning that asks: How should an agent act in an environment to maximize long-term reward through trial and error? I focus on households in the canonical consumption-savings model, Buffer Stock, so in this context the *agent* is the household, the *environment* is the Buffer-Stock model, and the *long-term reward* is discounted lifetime utility.

The application of reinforcement learning (RL) in economics is growing. Most of the literature focuses on reinforcement learning as a computational tool to approximate solutions to large models, where conventional methods are very slow or infeasible. However, little attention has been given to the idea of RL agents as *artificial economic agents*. The RL agent forms a policy based on its experience from interacting within the economic environment, and it may be more in line with how we think of real economic agents. Empirical evidence suggests that households are not perfectly rational, and this is typically modeled by including adjustment costs and biases explicitly in the household's problem. One hypothesis is that the RL household can capture this *bounded rationality* implicitly, because it is an imperfect problem solver by construction. In this thesis, I aim to shed some light on the potential of this idea by illustrating an RL household in the Buffer-Stock model.

I assume that this RL household learns as if it uses the *Deep Deterministic Policy Gradient algorithm* (Lillicrap et al., 2019). This is a *model-free* algorithm, implying that the household does not know the Buffer-Stock model dynamics and cannot use them to learn a policy. I describe the algorithm and what it implies about household learning. I then implement the algorithm in the finite-horizon Buffer Stock. First, I run *training* simulations, where the RL household lives through a number of life cycles in the Buffer-Stock environment to learn a policy. Second, I run *test* simulations, where the RL household executes the policy it converged to during training. Here, I compare its behavior to the optimal dynamic programming (DP) benchmark. To start, the RL household is trained and tested in a *baseline* Buffer-Stock environment. Then, I do a

number of parameter experiments, where I examine how the RL household adapts to changes in income risk and the interest rate.

The simulations show that the RL household tends to act rationally. In some cases, it exhibits near-optimal behavior, while it has more substantial deviations from it in other cases. Sub-optimal behavior may not be a problem in itself, but if reinforcement learning is to have merit as a model of a boundedly rational agent, deviations from optimal behavior must arguably have an economic interpretation. I.e., if the RL household learns a sub-optimal policy, what drove it? While there are *mechanical* reasons for this - one can point to various algorithm settings - the *economic* reasons remain somewhat of a "black box". I find this to be a weakness of the RL household.

The rest of this thesis is structured as follows. Section 2 covers related literature, and Section 3 presents the Buffer-Stock model. In Section 4, I describe how to solve the Buffer-Stock model with dynamic programming, and in Section 5 I describe the households as reinforcement learners, including the full learning algorithm. Section 6 compares RL households and DP households in life-cycle simulations. Section 7 discusses and Section 8 concludes. My code is available at Github. Running *par_exp.py* and *hyperpar_exp.py* reproduces all figures and tables.

2 Related Literature

A growing number of papers study the use of deep reinforcement learning[1] to approximate solutions with high accuracy to large models with heterogeneous agents and high-dimensional state spaces (see Fernández-Villaverde et al., 2024: Maliar et al., 2021: Azinovic et al., 2022: Han et al., 2022: Hill et al., 2021: Payne et al., 2024: Hall-Hoffarth, 2023: Kase et al., 2024). Conventional solution methods face the curse of dimensionality, where the time to solve the models grows exponentially in the number of state space dimensions. The idea is that approximating solutions with deep reinforcement learning can tame the curse of dimensionality and allow larger and more nuanced models.

The basic approach is to represent the policy function with a neural network and then train it to minimize or maximize an objective related to the model. For example, in a consumption-savings model in Maliar et al. (2021), the objective can be Euler errors, Bellman errors, or discounted lifetime utility. The neural network is trained on sample data from model simulations. First, the

[1] "Deep" means that multi-layered neural networks are used.

model is simulated with some policy function, and the data is inputted to the neural network, which evaluates the objective. Then, the weights of the neural network are updated to minimize or maximize the objective. Through iterations, the neural network can converge to the optimal policy function. For example, Maliar et al. (2021) solve a HANC model, and Kase et al. (2024) solve a HANK model.[2]

The idea behind this method is in very brief terms that (i) sampling data from model simulations avoids grids which are computationally expensive and (ii) that a neural network can approximate a wide range of functions (see universal approximation theorems in e.g. Hornik et al., 1989).

The training of the neural network, where the optimal policy function is approximated through an iterative process of model simulation and objective minimization/maximization, can be viewed as the *learning process* of an economic agent. The literature described so far primarily views this learning process as a means to approximate the optimal policy with high accuracy. Little attention has been given to whether sub-optimal learning outcomes can be relevant. One example is Shi (2023) who models RL households in a consumption-savings environment. She finds that the RL household's behavior can be shaped with a setting of the learning algorithm.

The idea of learning-based economic agents can be traced back to Holland and Miller (1991) and Arthur (1991), who argued for *artificial adaptive agents* that behave according to algorithms rather than explicit optimality conditions. In fact, Holland and Miller (1991) mention RL agents as a possibility, even though reinforcement learning was nascent at the time. The idea of artificial agents leans into agent-based models (ABM), in which a large number of heterogeneous economic agents interact within a simulated environment, following either adaptive or heuristic-based decision rules. One example of an ABM model is EURACE, which was an attempt to build a large agent-based model of the European economy Deissenberg et al. (2008).

3 The Buffer-Stock Model

I use the Buffer-Stock model (Deaton, 1991 and Carroll, 1997) as showcase for the RL household. Given that it is a dynamic model with stochastic income shocks, it is a sufficiently challenging environment for an RL household without being unnecessarily complex.

[2]Heterogeneous Agents New-Keynesian (HANK) and Heterogenous Agents Neo-Classical (HANC).

3.1 Model Description

The household chooses consumption to maximize their expected, discounted life-time utility

$$V(0, M, P) = \max_{\{C_t\}_{t=0}^{T-1}} \mathbf{E} \left(\sum_{t=0}^{T-1} \beta^t \frac{C_t^{1-\rho}}{1-\rho} \middle| M_0 = M, P_0 = P \right), \tag{1}$$

subject to the following conditions known as the *transition functions*

$$A_t = M_t - C_t \tag{2}$$

$$M_{t+1} = RA_t + Y_{t+1} \tag{3}$$

$$A_t \geqslant 0, \tag{4}$$

where the income process is

$$Y_{t+1} = \tilde{\xi}_{t+1} \cdot P_{t+1} \tag{5}$$

$$P_{t+1} = \psi_{t+1} \cdot P_t \tag{6}$$

$$\tilde{\xi}_{t+1} = \begin{cases} \mu & \text{with prob. } \pi \\ \frac{\xi_{t+1} - \pi\mu}{1-\pi} & \text{else} \end{cases} \tag{7}$$

$$\log \psi_{t+1} \sim \mathcal{N}(-0.5\sigma_\psi^2, \sigma_\psi^2) \tag{8}$$

$$\log \xi_{t+1} \sim \mathcal{N}(-0.5\sigma_\xi^2, \sigma_\xi^2) \tag{9}$$

Model variables. The main model variables are C_t; consumption, A_t; end-of-period assets, Y_t; income, and M_t; beginning-of-period cash on hand. (2) states that end-of-period assets are given by left-over cash. (3) states that cash-on-hand is given by income plus end-of-period assets times the gross interest rate R. The household faces a liquidity constraint given by (4).

Income process. The household's income is described by (5). It has a permanent and transitory component with the permanent in (6) and the transitory component in (7). The permanent component is persistent and follows a random walk, while the transitory component is temporary and have no persistence. Both components are driven by log-normal shocks, ψ and ξ, where construction implies $\mathbf{E}(\psi) = \mathbf{E}(\tilde{\xi}) = 1$ (7) states that "bad" transitory income shocks happen with probability π, in which case income growth is down-scaled with factor μ.

Model environment. The model implies the following dynamic *environment*: In each period, the household observes the *state* $\boldsymbol{s} = (t, M, P)$, were t, M, and P are known as the *state variables*. Then, the household chooses its *action variables*; consumption C_t, and end-of-period assets A_t. Since C_t is given by A_t and vice versa, using C_t as the action variable is sufficient. A consumption choice implies next-period state $\boldsymbol{s}' = (t+1, M', P')$. The consumption choice is made with the *policy function*, which maps the state variables to a consumption choice. To make a clear distinction between a consumption choice and the policy function, I denote the policy function π.

3.2 Normalization

I normalize model variables with respect to P_t as is standard in the Buffer-Stock model. Denote lower-case variables as normalized variables:

$$c_t = \frac{C_t}{P_t}, \quad a_t = \frac{A_t}{P_t}, \quad m_t = \frac{M_t}{P_t}$$

The model then reduces to

$$\begin{aligned} v(0, m) = \max_{\{c_t\}_{t=0}^{T-1}} \mathbf{E} & \left(\sum_{t=0}^{T} \beta^t \frac{c_t^{1-\rho}}{1-\rho} \,\middle|\, m_0 = m \right) \\ \text{s.t.} & \\ a_t &= m_t - c_t \\ m_{t+1} &= \frac{R}{\psi_{t+1}} a_t + \tilde{\xi}_{t+1} \\ a_t &\geqslant 0, \end{aligned}$$

with shocks still specified by (7)-(9). Normalization has the benefit that it reduces the dimensionality of the model in the sense that the policy function now only depends on time period and cash on hand, i.e. $\pi(t, m)$ instead of $\pi(t, M, P)$. Firstly, this makes it less expensive to solve the household problem using DP (this becomes clear in Section 4). Secondly, it makes the policy function easier to illustrate, as it can be plotted in a two-dimensional *m-c* diagram, and thus differences between the RL household's policy function and the optimal policy function become more clear.

3.3 Optimal Policy

The optimal policy function is concave, implying that the household saves as cash-on-hand increases. This is *precautionary* saving - even though the household receives income in every period (and there is no retirement), it still saves because future income is uncertain. Thus, the function of precautionary savings is to guard the household against adverse income shocks. They arise due to a combination of stochastic income shocks, prudence in utility ($u''' > 0$), and the liquidity constraint ($a_t \geqslant 0$).

Prudent utility. If utility is not prudent ($u''' = 0$, e.g. quadratic utility), there is no incentive for precautionary savings. The intuitive reason is that no prudence implies linear marginal utility. Any decrease in utility resulting from a negative income shock is perfectly offset by a positive income shock of the same magnitude, because marginal utility is just proportionally higher. However, if utility is prudent, marginal utility is concave. The "punishment" for low consumption becomes greater, and therefore the household saves to avoid very low consumption.

Liquidity constraint. The liquidity constraint prevents the household from using borrowing to smooth consumption. But it also leads to hand-to-mouth behavior for low cash-on-hand. In absence of liquidity constraints, the Euler equation is

$$u'(c_t) = R\beta\mathbf{E}_t(u'(c_{t+1})).$$

However, with a borrowing constraint, the household may not have adequate liquidity to satisfy the Euler equation. If the constraint is binding, the household cannot consume to the degree it wants in period t, and the condition becomes

$$u'(c_t) \geqslant R\beta\mathbf{E}_t(u'(c_{t+1})),$$

The best the household can do is to consume hand-to-mouth. The intuition is that it can afford this because it anticipates higher income in the future.

3.4 Terminology

As there is no analytical solution to the Buffer-Stock model, dynamic programming is used to solve it. Reinforcement happen to use much of the same terminology, as both belong to *optimal control theory* (Sutton and Barto, 2018). I set the stage by outlining the necessary terminology.

Value function. The *value function* $v_\pi(t, m)$ is the utility stream remaining in period t that can be expected from following the policy π, given some $m_t = m$:

$$v_\pi(t, m) = \mathbf{E}_\pi \left(U_t \mid m_t = m \right)$$
$$U_t = u_t + \beta u_{t+1} + \cdots + \beta^{T-t} u_T = \sum_{k=t}^{T} \beta^{k-t} u_k.$$

The value function is useful, because it connects the policy to utility streams. The household is concerned with what utility streams can be expected from what policies, which the value function expresses. Note that expected life-time utility for some starting $m_0 = m$ is analogous to $v_\pi(0, m)$. Therefore, the household's problem can be solved by maximizing the value with respect to the policy function, which defines the *optimal value function* and *optimal policy function*:

$$v^*(t, m) = \max_\pi v_\pi(t, m)$$
$$\pi^* = \arg\max_\pi v_\pi(t, m)$$

Both dynamic programming and the RL algorithm used in this thesis aim to estimate the value function, and then use it to choose the policy function that maximizes value. However, they do it in different ways.

Bellman equation. The *Bellman equation* expresses the value function on a recursive form. Below, the Bellman equation is stated for the optimal value function v^*, sometimes known as the Bellman *optimality* equation.

$$\underbrace{v^*(t, m)}_{\text{Current value}} = \max_c \Big[\underbrace{u(c)}_{\text{Immediate utility}} + \underbrace{\beta \mathbf{E} v^*(t+1, m')}_{\text{Continuation value}} \Big]$$
$$= \max_c \left[u(c) + \beta \int_{\tilde{\xi}'} \int_{\psi'} v^*(t+1, m') \, f(d\tilde{\xi}', d\psi') \right]$$

where

$$m' = \frac{R}{\psi'}(m - c) + \tilde{\xi}', \quad m - c \geqslant 0$$

Thus, the Bellman equation breaks the current value into immediate utility and a *continuation value*, i.e. the expected future discounted value. The continuation value depends on the next state. The next time period $t+1$ is clearly known as it is deterministic, but next cash-on-hand m' is stochastic due to income shocks. Therefore, an expectation is taken over the income shocks. This is an integral, because the income shocks follow continuous distributions.

To put things into perspective, the Bellman optimality equation is the mathematical formulation of Bellman's powerful principle of optimality: "An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision" (Bellman, 1957). Dynamic programming is a method to compute a numerical solution to the Bellman equation. The solution is the optimal value function v^*, and the optimal policy function π^* can then be extracted from the argument as shown previously.

However, the Bellman equation it is not used in the RL algorithm in this thesis, because the household is assumed to be a *model-free* reinforcement learner. It does not know the model's transition functions and therefore cannot explicitly compute the expectation. For example, it would require knowledge of the income shock distributions to compute the integrals.

State and action values. v_π is also called a *state-value* function, as it takes the state $\boldsymbol{s} = (t, m)$ as input. There is also an *action-value* function $q_\pi(t, m, c)$ that takes both state and action (consumption) as input. The action value function is the utility stream remaining in period t that can be expected given that the household chooses current consumption $c_t = c$ and thereafter follows policy π:

$$q_\pi(t, m, c) = \mathbf{E}_\pi \left(U_t \mid m_t = m, c_t = c \right)$$

Dynamic programming relies on state values, while the RL algorithm relies on action values. The reason is that the RL household cannot explicitly compute the expectation as a model-free reinforcement learner, as explained above. This makes it difficult for the RL household to predict

the consequences of its consumption choices. For example, suppose that the RL household finds itself in state $\boldsymbol{s}$ and wants to predict the value of consuming c in the next period. With state values, the requires the computation of $\mathbf{E}v(t+1, m')$, which is not possible for the model-free RL household. However, it is straightforward to make the prediction using action values $q(t, m, c)$ by plugging in c.

4 Solution with Dynamic Programming

4.1 Discretization

To facilitate a numerical solution with dynamic programming, some continuous variables must be discretized. First, the state space for cash-on-hand m is discretized. The true domain is unbounded above, because there is in principle no limit to how much wealth the household can hold. I choose a sufficiently large upper limit so that ignoring very high values does not significantly alter the optimal policy (specific values are reported in Section 4.3). The natural lower bound is 0.

Because dynamic programming uses the Bellman equation directly, some numerical integration is required to compute the expectation. I use *quadrature*, specifically Gauss-Hermite quadrature (Judd, 1998). The core idea of quadrature is to approximate an integral by summing weighted function evaluations at specific points. The result is a discrete set of shock realizations $\tilde{\xi}^i$ and ψ^i with associated probabilities $p^i_{\tilde{\xi}}$ and p^i_{ψ}. The grids are summarized below.

$$\mathcal{G}_m = \left\{ m^i \right\}_{i=0}^{N_m}, \quad \mathcal{G}_{\tilde{\xi}} = \left\{ (\tilde{\xi}^i, p^i_{\tilde{\xi}}) \right\}_{i=0}^{N_{\tilde{\xi}}}, \quad \mathcal{G}_{\psi} = \left\{ (\psi^i, p^i_{\psi}) \right\}_{i=0}^{N_{\psi}}$$

With the grids above, the Bellman optimality equation can now be discretized and thus solved numerically:

$$v^*(t, m) = \max_{c \in [0, m]} \left[u(c) + \beta \sum_{i=0}^{N_{\tilde{\xi}}} \sum_{j=0}^{N_{\psi}} p^i_{\tilde{\xi}} \cdot p^i_{\psi} \cdot v^*(t+1, m'_{i,j}) \right]$$

$$m'_{i,j} = \frac{R(m-c)}{\psi^j} + \tilde{\xi}^i, \quad (m-c) \geqslant 0$$

While cash-on-hand m is discretized, the consumption choice c is not. The implication is that consumption choices likely result in next-period cash-on-hand m' that is not "on the grid". This becomes a challenge when evaluating the value function, which is only computed at discretized grid points of m. To find values in between grid points of m, linear interpolation is used as an approximation.

4.2 Backwards Induction

The idea is to solve the discretized Bellman optimality equation for each point (t, m) in the state space grid:

$$\{0, ..., T-1\} \times \mathcal{G}_m = \left\{ (t, m) \,\middle|\, t \in \{0, \ldots, T-1\},\ m \in \mathcal{G}_m \right\}$$

This is a meaningful goal, because the solution to the Bellman optimality equations is the optimal value function v^*, and by definition its argument is the optimal policy function π^*, which solves the household's problem, as covered in Section 3.4. Because the horizon is finite ($T < \infty$), backwards induction can be used to solve the discretized Bellman optimality equations by moving backwards in time starting from the final period $t = T-1$ and iterating back to the initial period $t = 0$. The outer loop is over t, and the inner loop is over m.

For example, in the final period, optimal consumption is simply to consume everything regardless of current cash-on-hand, i.e. value $v(T, m) = u(m)$ and policy $\pi^*(T, m) = m$ for all $m \in \mathcal{G}_m$. Now, that $v(T, m)$ is known, $v(T-1, m)$ and $\pi^*(T-1, m)$ can be found for all $m \in \mathcal{G}_m$ by computing the right hand side of the Bellman equation. The same principle is applied for $T-2$, $T-3$ and so on. See Algorithm 1 for pseudo code for backwards induction.

If the state space had additional dimensions, additional loops would have to be added. For example, in the non-normalized Buffer-Stock model, the state includes permanent income P, and then the Bellman optimality equation would have to be solved for each point (t, P, M). This *nested* loop structure creates a *curse of dimensionality*, where the time to solve the household problem grows exponentially in the number of state space dimensions.

4.3 Implementation

I use code from Jeppe Druedahl accessible from Github.[3] I use the pre-set grid settings in this code. I.e., a non-linear grid $\mathcal{G}_m$, with preset range $[10^{-6}, 20]$ and 600 grid points. The non-linear grid ensures greater density closer to 0. For quadrature, 6 nodes is used for both income shocks.

The code solves the non-normalized Buffer-Stock model, and therefore I have modified it slightly to the normalized version. The code implements a slight modification of the plain-vanilla backwards induction algorithm I present below, which speeds it up. For example, the the numerical optimization is solved with golden-section search, instead of looping over every m in grid $\mathcal{G}_m$ to find $c_{\max}$, golden-section search is applied. In addition, the continuation value w is computed with arrays instead of loops.

It is possible to make backwards induction more efficient by using so-called *post-decision functions* to compute the continuation values w or the *endogenous grid point method* (Carroll, 2006) to find c_{opt} with the Euler equation. I have chosen to stick with the plain-vanilla backwards induction for the sake of simplicity to place more focus on reinforcement learning.

[3] Github.com/NumEconCopenhagen/ConsumptionSavingNotebooks

Algorithm 1: Backward Induction

Input: Grids $\mathcal{G}_m$, $\mathcal{G}_{\tilde{\xi}}$, $\mathcal{G}_\psi$; model parameters T, β, ρ, R; tolerance ϵ_v

Initialize $v[:, :]$
Initialize $c[:, :]$
foreach $t \in \{T-1, \ldots, 0\}$ **do**
  foreach $m \in \mathcal{G}_m$ **do**
    if $t = T$ **then**
      $c_{\text{opt}} \leftarrow m$
      $v_{\text{max}} \leftarrow \frac{c^{1-\rho}}{1-\rho}$
    else
      $c_{\text{opt}} \leftarrow 0$
      $v_{\text{max}} \leftarrow -\infty$
      foreach $c \in \mathcal{G}_m$ **do**
        $c \leftarrow \min(m, c)$
        $a \leftarrow m - c$
        $w \leftarrow 0$
        foreach $(\tilde{\xi}, p_{\tilde{\xi}}) \in \mathcal{G}_{\tilde{\xi}}$ **do**
          foreach $(\psi, p_\psi) \in \mathcal{G}_\psi$ **do**
            Next-period cash on hand: $m' \leftarrow \frac{Ra}{\psi} + \tilde{\xi}$
            $w_{\tilde{\xi},\psi} \leftarrow$ Interpolate m' on $v[t+1, :]$
            $w \leftarrow w + p_{\tilde{\xi}} \cdot p_\psi \cdot w_{\tilde{\xi},\psi}$
        $v \leftarrow \frac{c^{1-\rho}}{1-\rho} + \beta \cdot w$
        if $v - v_{max} > \epsilon_v$ **then**
          $v_{\text{max}} \leftarrow v$
          $c_{\text{opt}} \leftarrow c$
    $c(t, m) \leftarrow c_{\text{opt}}$
    $v(t, m) \leftarrow v_{\text{max}}$
return v, c

5 Households as Reinforcement Learners

A fundamental difference between dynamic programming and reinforcement learning is that dynamic programming is *grid-based*, while reinforcement learning is *simulation-based*. With dynamic programming, the optimal policy function is computed by finding optimal consumption for each point in the state space grid. Reinforcement learning takes a different approach by simulating the model to collect data, and this data is then used to estimate the optimal policy function. Reinforcement learning can be thought as *approximate* in nature, and dynamic programming as *exact* in nature.[4]

Simulating data has a computational advantage, because it avoids grids tensor grids that are expensive, because they imply nested loops, as explained in Section 4. This is one motivation for the use of reinforcement learning - a method to approximate solutions to large, high-dimensional models, where grid-based methods like dynamic programming become very slow or even infeasible. But simulating data also introduces sample noise, which is why RL algorithms often are not guaranteed to converge to the optimal policy function contrary to DP algorithms. Still, reinforcement learning shows promising results as a computational tool to accurately approximate solutions to economic models (see Section 2).

The goal of reinforcement learning as a tool to approximately solve models is *performance* - to get as close to the optimal policy as possible. But can any economic sense be made of convergence to sub-optimal policies? We may think of economic agents as RL agents; imperfect problem solvers that may deviate from perfectly optimal behavior. In this section, I illustrate this idea by describing how an RL algorithm applied to the Buffer-Stock can be interpreted as the household interacting within the Buffer-Stock environment and using its experience to learn a policy. I focus on a single algorithm as a showcase, though there are many kinds of RL algorithms (see e.g. Sutton and Barto, 2018). This algorithm is the *Deep Deterministic Policy Gradient* (DDPG) algorithm (Lillicrap et al., 2019). This is chosen, because it assumes continuous state and action spaces, which aligns with continuous spaces for cash-on-hand and consumption in the Buffer-Stock model.

[4] In general, reinforcement learning does not require simulations, but in this context it is the relevant data source. The alternative is data from real-world interactions.

5.1 General Framework

The RL household holds a value function q_π and a policy function π. If the true value function was known, the optimal policy could easily be found. But naturally, it is not. Therefore, the household must estimate the value function and use the estimate to choose its policy.

In the RL algorithm, the household lives through E life cycles, where it iteratively updates its value function and policy function. This is referred to as the *training* or *learning*. The very core idea of reinforcement learning is that behavior that tends to produce high utility levels will be associated with high value, and this steers the policy function towards that behavior. In this way, rewarding behavior is *reinforced* - an idea that in fact has biological connections to how human and animals learn (Sutton and Barto, 2018).

Here, I present the *simplified* RL algorithm. The simplified RL algorithm has exactly the same structure as the full RL algorithm but is stripped of implementation details and better illustrates the main ideas. The full algorithm is then presented in the Section 5.4.

Simplified RL algorithm:

For each e in $\{0, \ldots, E-1\}$ life cycles:

For each t in $\{0, \ldots, T-1\}$ periods:

- Observes the current state $\boldsymbol{s} = (t, m)$ and choose:

$$c = \text{clip}\left(\pi(\boldsymbol{s}) + \eta, 0, m\right), \quad \eta \sim \mathcal{N}(0, \sigma_c^2 \cdot e^{-\gamma e}) \tag{10}$$

  Observe utility u and next state $\boldsymbol{s'} = (t+1, m')$.

- Compute the *temporal difference error* (TD error):

$$\delta = \underbrace{u + \beta \cdot q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}'))}_{\text{Semi-observed value}} - \underbrace{q_\pi(\boldsymbol{s}, c)}_{\text{Predicted value}} \tag{11}$$

$$q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}') = 0 \quad \text{if } t = T-1$$

- Update the value function to minimize the squared TD error:

$$\min_{q_\pi} \delta^2 \tag{12}$$

- Update the policy function to maximize value:

$$\max_{\pi} q_{\pi}(\boldsymbol{s}, \pi(\boldsymbol{s})). \tag{13}$$

Value function update. The key component of the algorithm is the temporal difference error (TD error) in (11). The TD error is the difference between a semi-observed value and a predicted value (semi-observed because u is the only real feedback). The *error* arises if the household has inaccurate value predictions. A positive TD error means that the consumption choice c was "good" because it led to a state with a better-than-expected value. A negative TD error means that the consumption choice c was "bad" because it led to a state with a worse-than-expected value. Thus, the TD error guides the household's value function updates:

- If $\delta > 0$, the semi-observed outcome was better than expected, so increase $q_{\pi}(\boldsymbol{s}, c)$.
- If $\delta < 0$, the semi-observed outcome was worse than expected, so decrease $q_{\pi}(\boldsymbol{s}, c)$.
- If $\delta = 0$, the value estimate was accurate, so no update is needed.

The TD error is biologically connected to human learning (Sutton and Barto, 2018). The neurotransmitter *dopamine* spikes when a reward is received, signaling an unexpected good outcome, and conversely if a reward is missing, dopamine levels decrease. This is analogues positive and negative TD errors. It is beyond the scope of this thesis to cover the connection between reinforcement learning and biological learning. And just because reinforcement learning is connected to biological learning, it does not necessarily make an accurate model of how economic households learn and behave. But after all, economic agents are human, and their behavior is arguably biologically rooted to some degree.

Policy function update. Relative to the value function update, the policy function update is more straightforward. The household chooses the policy that maximizes future predicted value (13). This is an intuitive goal, since the household aims to maximize remaining lifetime utility (i.e. value).

Putting it all together, the RL algorithm has the following interpretation. The TD errors serve as learning signals; if the squared TD errors are positive, it indicates that the household has inaccurate value predictions - its understanding of the environment is lacking. It responds by updating its value function, which in turn guides the policy function to what it perceives as rewarding

behavior. The RL household is rational and forward-looking, because it chooses its policy to maximize value. But it is biased by its experience and has a limited capacity, because it must learn the value of policies by associating different state-action pairs with different outcomes.

Exploration. If the household sticks to its policy at all times, it will likely learn a suboptimal policy, because it will be "unaware" of what value other policies can offer. More precisely, it will leave parts of the state-action space unexplored, and its value predictions in these parts will likely be inaccurate. But the true value may be higher in these parts, and there may be a better policy. Therefore, the household occasionally deviates from its current policy reflected by the noise in (10). This is called *exploration*. Note that due to the liquidity constraint and non-negative consumption, the consumption choice is clipped between 0 and m.

For a concrete example of exploration, suppose that the household chooses some $c \neq \pi(\boldsymbol{s})$. If the TD error is positive, it signals that consuming c in state $\boldsymbol{s}$ yields higher value than what was predicted under the current policy, i.e. the current policy is inferior at point $(\boldsymbol{s}, c)$. The household responds by increasing its value estimate at point $(\boldsymbol{s}, c)$ and the policy function to favor c in state $\boldsymbol{s}$.

The noise has an initial starting standard deviation of σ_c, which then decays exponentially in the number of training life cycles with the rate γ. It is high early on to allow the household to explore different policies, and low in late stages to facilitate converge to the best policy the household has found.

5.2 Model-free and Model-based Households

In this thesis, the households are modeled as *model-free* reinforcement learners. This means that they do not know the economic model. In particular, it implies that the households use *sample* transitions in the temporal difference contrary to *expected* transitions when computing the temporal difference:

$$u + \beta \cdot \underbrace{q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}'))}_{\text{Using sample transition}} - q_\pi(\boldsymbol{s}, c)$$

$$u + \beta \cdot \underbrace{\mathbf{E}\big[q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}'))\big]}_{\text{Using expected transition}} - q_\pi(\boldsymbol{s}, c) = u + \beta \cdot \int_{\psi'} \int_{\tilde{\xi}'} q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}'))\, f(d\tilde{\xi}', d\psi') - q_\pi(\boldsymbol{s}, c)$$

Sample transitions are used in following way: After making a consumption choice, the household observes the next state $\boldsymbol{s}'$ and then uses this to estimate the continuation by plugging it into its current estimate of the value function $q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}'))$. In this way, the household performs a Monte Carlo style integration to estimate the continuation value. On the other hand, if the household uses expected transitions, it would not rely on an observation of the next state $\boldsymbol{s}'$. Instead, the household would consider all possible next states using the model's income shock distributions and other transition functions to compute the continuation value - just like the DP household. However, the as a model-free reinforcement learner, they household cannot compute the expectation directly in this way.

What are the implications of assuming a model-free household? One point is that although expected transitions are more expensive computationally, they are more accurate, because they do not suffer from a sampling error. Expected transitions can thus increase accuracy of the value function estimate.

A second point relates to how the household adapts to changes in the environment - this could be any changes to the model transition functions, e.g. a tax rate on income or an increase in income risk. A model-based household can adapt without needing much direct experience. Since the model-based household knows the model, any environment changes will lead to changes in the expectation operator, even before the household gathers any new experience in the changed environment. On the other hand, for a model-free household to adapt, it must visit different states, consume and directly experience the outcomes many times, before it adjusts it value and policy functions accordingly.

In this sense, the model-based household is more forward-looking, while the model-free household is more habitual. Assuming a model-free household may be a too strict assumption. While it may be reasonable to assume that the household does not know the exact income shock distributions, it seems unreasonable to assume that the household does not know the basic transition that savings are given by cash-on-hand subtracted by consumption (2).

5.3 Neural Networks

Since the state space for m is continuous, the household must generalize across different values of m. Therefore, the value and policy functions are parameterized

$$q_\pi^\theta(\boldsymbol{s}, c), \quad \pi^\Phi(\boldsymbol{s}),$$

where θ and Φ are the vectors of parameters that define these functions. For simplicity, I omit superscripts in subsequent notation. To approximate q_π and π, I assume that households use *neural networks*, following the original DDPG algorithm. q_π is referred to as the *value network*, and π is referred to as the *policy network*. Given that the Buffer-Stock environment is relatively simple, neural networks are arguably not necessary. However, they make life easier, in the sense that it is not necessary to specify a functional form as would be required for e.g. linear approximation.

A neural network is a flexible functional approximator that maps input variables to output variables like any other mathematical function. "Flexible" means that neural networks can accurately approximate a wide range of functions, which proven in a number of universal approximator theorems (Hornik et al., 1989). The value network takes state and consumption $(\boldsymbol{s}, c)$ as input and outputs an associated value $q_\pi(\boldsymbol{s}, c)$. The policy network takes state $\boldsymbol{s}$ as input ad outputs an associated policy $\pi(\boldsymbol{s})$.

The transformation from input to output happens the network's *hidden layers* which consist of *neurons*. When an input is passed to the network, it "flows" through each neuron in each of the hidden layers. In this thesis, it suffices to know that each neuron has a *weight* and an *activation function* that shape the output of the neural network.[5] The weights are analogues to the parameters θ and Φ. This means that when the household updates its value and policy functions in, it updates the network weights θ and Φ.

The updates are done with *gradient descent* with the objective functions being the squared temporal difference (12) for the value network and the predicted value (13) for the policy network. The gradients of the networks are computed with *backpropagation*, a method that uses the chain rule to differentiate effectively. Updates of the value and policy networks are summarized below.

[5] See Goodfellow et al. (2016) for more information on neural networks.

Update of value and policy networks:

1. Evaluate the objective function $L(\theta)$ or $J(\boldsymbol{\phi})$:

$$L(\theta) = \delta^2 = \left(u + \beta \cdot q_\pi(\boldsymbol{s}', \pi(\boldsymbol{s}')) - q_\pi(\boldsymbol{s}, c)\right)^2, \qquad J(\boldsymbol{\phi}) = q(\boldsymbol{s}, \pi(\boldsymbol{s}))$$

2. Compute the gradients of the objective function:

$$\nabla_\theta L(\theta) = \frac{\partial L}{\partial \theta}, \quad \nabla_\Phi J(\boldsymbol{\phi}) = \frac{\partial J}{\partial \boldsymbol{\phi}}$$

3. Use gradient descent to update the parameters:

$$\theta \leftarrow \theta - \alpha_\theta \nabla_\theta L(\theta), \quad \boldsymbol{\phi} \leftarrow \boldsymbol{\phi} - \alpha_\Phi \nabla_\Phi J(\boldsymbol{\phi}).$$

α_θ and α_Φ in step 3 are the *learning rates* that adjust the step size of the gradient descent. If the learning rate is *high*, the network updates become large, which can cause oscillation around the global minimum or divergence away from it. If the learning rate is *low*, the network updates are small, which can lead to slow convergence or getting trapped in local minima.

The learning rates are among the *hyperparameters* of the RL algorithm. Hyperparameters are a kind of *algorithmic settings* in machine learning that affect how the household learns. Other hyperparameters include the number of hidden layers, the number of neurons per layer, the choice of activation functions, and initial weights of the neural network (the initial values in θ and $\boldsymbol{\phi}$). I postpone the choice of hyperparameter values to Section 5.4.

In practice, the value and policy networks are implemented using PyTorch in Python, which performs backpropagation efficiently with automatic differentiation. Gradient descent is done using PyTorch's built-in Adam optimizer, which is more efficient compared to the plain-vanilla gradient descent written above. To ensure that the liquidity constraint is not violated by the policy network, its final layer uses a Sigmoid activation function, which binds the output between 0 and 1. The final consumption level is then achieved by multiplying with the given m.

5.4 Implementation

This section builds on the *simple* RL algorithm to present the *full* RL algorithm. The core idea and structure is unchanged, but it includes a number of "add-ons" that I describe below. I found that an algorithm without these elements led to poor learning. Pseudo code for the full algorithm can be found in Algorithm 2 and 3.

Batch updates. When the household updates its value and policy functions, it uses transitions $(\boldsymbol{s}, c, u, \boldsymbol{s}')$. Instead of using only the most recent transition to update, *multiple* transitions are used. The household stores the transitions it experiences in a *replay buffer* $\mathcal{D}$, and when it updates its value and policy functions, it uses a *batch* of transitions $\mathcal{B}$, drawn randomly from the replay buffer:

$$\mathcal{B} = \{(\boldsymbol{s}^j, c^j, u^j, \boldsymbol{s}'^j)\}_{j=1}^{N_{\mathcal{B}}} \sim \mathcal{D}$$

Following Mnih et al. (2015), the idea of batches breaks correlations between consecutive experiences and thus lowers the variance of value and policy function updates. The intuition is that learning becomes more stable and less influenced by the most recent transitions.

Target functions. The RL algorithm suffers from a "moving target". This means that the minimum/maximum of the objective functions in (12) and (13) are moving during between iterations, because the policy and value networks are updated. If these updates are large, the temporal difference can change too rapidly from one iteration to the next and consequently the minimum/maximum moves aggressively. This can cause oscillations and divergence in the policy. To prevent this, Mnih et al. (2015) propose *target functions*, $\tilde{q}_\pi$ and $\tilde{\pi}$, with parameters $\tilde{\theta}$ and $\tilde{\phi}$. These are initialized as copies of the actual value and policy networks but evolve slowly:

$$\tilde{\theta} \leftarrow \tau\theta + (1-\tau)\tilde{\theta}, \quad \tilde{\phi} \leftarrow \tau\phi + (1-\tau)\tilde{\phi}.$$

where τ determines the "lack" in the target networks. The actual policy and value networks are then replaced with the corresponding target networks in the target of the temporal difference, i.e.

$$\min_{\theta} \left(\underbrace{u + \beta \cdot \tilde{q}_\pi(\boldsymbol{s}', \tilde{\pi}(\boldsymbol{s}'))}_{\text{Target}} - q_\pi(\boldsymbol{s}, c) \right)^2$$

The idea is that the target in the temporal difference remains more stable since the target functions evolve slowly.

Utility scaling. I found that the value and policy gradients tended to "explode", if utility was not clipped to stay within a certain range. This is likely because CRRA utility approaches negative infinity, when consumption approaches 0, which in turn can cause large gradients in the value and policy updates. Therefore, I clip utility as stated below. Clipping gradients is another way of preventing them from exploding.

$$u_t = \begin{cases} \overline{u} & \text{if } u(c) > \overline{u} \\ u(c) & \text{if } \underline{u} \leqslant u(c) \leqslant \overline{u} \\ \underline{u} & \text{if } u(c) < \underline{u} \end{cases}$$

where $\underline{u} = -5$ and $\overline{u} = 5$. Since I assume $\rho = 1$, utility rarely falls beyond these values. Note that utility is only clipped *during training*. In test simulations, utility is not clipped so that lifetime utility can be compared between RL and DP households.

Hyperparameters. I found that hyperparameter settings had substantial impact on the household's learning outcome (I document this in the Appendix). Therefore, I hand-tune the hyperparameters to a *baseline* Buffer-Stock environment defined in the following section and do not change them thereafter.

The hand-tuning is done by iterating the following process:

1. Select hyperparameters.
2. Run Algorithm 2 and save the learned policy.
3. Simulate a life-cycle of 10,000 households using the learned policy.
4. Calculate mean life-time utility $\bar{U}_0 = \frac{1}{N} \sum_{i=0}^{N} U_{0,i}$.

The final choice of hyperparameters is given by the selection that yields the highest mean life-time utility. I summarize the choice of hyperparameters and network architecture below.

Each network (value, policy and their targets) has two hidden layers, each containing 64 neurons that use the Rectified Linear Unit (ReLU) activation function. The networks are initialized with random weights. The main point is that the value and policy networks should be large enough

to be able to accurately represent the true value and policy functions, yet larger networks overfit more easily and require more training. I found best performance with the settings above.

The learning rate for the value network is set to $\alpha_\theta = 10^{-4}$, and the learning rate for the policy network is set to $\alpha_\phi = 0.5 \cdot 10^{-4}$. The target networks are updated with $\tau = 0.005$. The replay buffer has a capacity of $\mathcal{D}_{\text{max}} = 10^6$, with each batch consisting of $N = 256$ transitions. For exploration, noise is added to the policy with an initial standard deviation of $\sigma_c = 0.2$, which decays exponentially at a rate of $\gamma = 0.01$.[6]

Baseline environment. The baseline environment is defined by the following Buffer-Stock model parameters. The household's discount factor is set to $\beta = 0.96$, and the elasticity of substitution is $\rho = 1.0$. The gross interest rate is $R = 1.03$. Log-normal income shocks have standard deviations $\sigma_\xi = \sigma_\psi = 0.1$. Adverse transitory shocks occur with probability $\pi = 0.1$ and take the value $\mu = 0.5$. Initial cash-on-hand is drawn uniformly from the interval $[\underline{m}, \overline{m}] = [0, 2]$.

[6]While numbers like 64 neurons and a batch size of 256 are often chosen because they are powers of two, these parameters are not inherently required to be powers of two. But it can lead to more efficient memory usage and computation on GPUs. However, I am running everything on a CPU.

Algorithm 2: Household Reinforcement Learning Algorithm

Input: Model parameters T, $\overline{m}$, $\underline{m}$, β, ρ, R; shock grids $\mathcal{G}_\psi$, $\mathcal{G}_{\tilde{\xi}}$; number of episodes E; hyperparameters τ, α_θ, α_ϕ, σ_c, γ, N;

$\mathcal{D} \leftarrow \text{deque}(\text{maxLen} = \mathcal{D}_{\text{max}})$ `// replay buffer`

initialize $\tilde{\pi}(t, m; \tilde{\phi})$ `// target consumption function`

initialize $\tilde{q}_\pi(t, m, c; \tilde{\theta})$ `// target value function`

initialize $\pi(t, m; \phi)$ `// consumption function`

initialize $q_\pi(t, m, c; \theta)$ `// value function`

$\tilde{\phi} \leftarrow \phi$

$\tilde{\theta} \leftarrow \theta$

for $e \leftarrow 0$ **to** $E - 1$ **do**

  $m' \leftarrow \text{uniformNormalSample}(\text{min} = \bar{m}\,, \text{max} = \underline{m})$

  for $t \leftarrow 0$ **to** $T - 1$ **do**

    `// interactive step`

    $m \leftarrow m'$

    $\eta \leftarrow \text{normalRandomSample}(\text{mean} = 0\,, \text{var} = \sigma_c^2 \cdot \exp(-\gamma \cdot e))$

    $\psi \leftarrow \text{randomSample}(\mathcal{G}_\psi[:, 0]\,, \text{probs} = \mathcal{G}_\psi[:, 1])$

    $\tilde{\xi} \leftarrow \text{randomSample}(\mathcal{G}_{\tilde{\xi}}[:, 0]\,, \text{probs} = \mathcal{G}_{\tilde{\xi}}[:, 1])$

    $c \leftarrow \text{clip}\,(\pi(t, m) + \eta\,, 0\,, m)$

    $u \leftarrow \text{clip}(c^{1-\rho}/(1-\rho)\,, \underline{u}\,, \overline{u})$

    $a \leftarrow m - c$

    $m' \leftarrow \frac{Ra}{\psi} + \tilde{\xi}$

    $\mathcal{D}.\text{appendLeft}([t\,, m\,, c\,, u\,, t + 1/(T-1)\,, m'])$

    `// update step`

    if $\mathcal{D}.len() > N$ **then**

      $\mathcal{B} \leftarrow \text{uniformRandomSample}(\mathcal{D}\,, \text{size} = N)$

      $(\theta\,, \phi\,, \tilde{\theta}\,, \tilde{\phi}) \leftarrow \text{learningStep}(\beta\,, t\,, \mathcal{B}\,, \tilde{q}_\pi\,, \tilde{\pi}\,, q_\pi\,, \pi)$ `// see Algorithm 3`

return $(\tilde{\theta}\,, \tilde{\phi})$

Algorithm 3: learningStep

Input: Model parameters β; time t; replay buffer $\mathcal{B}$; target functions $\tilde{q}_\pi$, $\tilde{\pi}$; value function q_π; policy function π; hyperparameters α_θ, α_ϕ

$(t_\mathcal{B}, m_\mathcal{B}, c_\mathcal{B}, u_\mathcal{B}, t'_\mathcal{B}, m'_\mathcal{B}) \leftarrow \mathcal{B}(:, 0{:}5)$

`// value function`

$c'_\mathcal{B} \leftarrow \tilde{\pi}(t'_\mathcal{B}, m'_\mathcal{B})$

$w_\mathcal{B} \leftarrow (t < 1)\, \beta\, \tilde{q}_\pi\big(t'_\mathcal{B}, m'_\mathcal{B}, c'_\mathcal{B}\big)$

$L_\mathcal{B} \leftarrow (q(t_\mathcal{B}, m_\mathcal{B}, c_\mathcal{B}) - u_\mathcal{B} - w_\mathcal{B})^2$

$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta L$

`// policy function`

$J \leftarrow q(t_\mathcal{B}, m_\mathcal{B}, \pi(t_\mathcal{B}, m_\mathcal{B}))$

$\phi \leftarrow \phi + \alpha_\phi \nabla_\phi J$

`// target functions`

$\tilde{\theta} \leftarrow \tau\, \theta + (1 - \tau)\, \tilde{\theta}$

$\tilde{\phi} \leftarrow \tau\, \phi + (1 - \tau)\, \tilde{\phi}$

return $(\theta, \phi, \tilde{\theta}, \tilde{\phi})$

6 Life-cycle Simulations

6.1 Setup

In this section, I compare compare DP and RL households through life-cycle simulations. The procedure is as follows. The households face a given environment, defined by a set of model parameters. The DP policy is computed using backwards induction (Algorithm 1), and the RL policy is computed using the RL algorithm (Algorithm 2). These policies are then used to simulate two identical samples of households, the only difference being the policy they use to make their consumption-saving decisions.

Simulation procedure:

1. Define the environment by choosing model parameters.
2. Initialize value and policy functions v_{RL}, π_{RL}, v_{DP}, π_{DP}.
3. Find policies by running algorithms with model parameters from step 1 as input:
   (a) RL policy: Run Algorithm 2 and save returned π_{RL}.
   (b) DP policy: Run Algorithm 1 and save returned π_{DP}.
4. Simulate life cycles in the environment:
   (a) RL sample uses π_{RL} to make its consumption choices.
   (b) DP sample uses π_{DP} to make its consumption choices.

First, note that the policies depend on the environment. This means that I re-run the algorithms, if a model parameter is changed. Second, note that the entire sample of RL households uses the same policy function resulting from the training in step 3a above. However, the households will differ in their initial cash-on-hand and face different income shocks throughout their lifetime. Thus, there is no heterogeneity in learning among the RL households, but in terms of initial endowment and income.

I consider a sample size of $N_{HH} = 10{,}000$ households, and each household lives through one life cycle of $T = 20$ periods. These values are chosen given the trade-off between a large sample size and computational expenses. The RL household is trained for $E = 2{,}000$ life cycles. This

value is chosen such that any additional training only produces negligible changes in the policy (meaning that the RL household has converged to a policy).

First, I consider simulations the *baseline* environment defined earlier in Section 5.4. Next, I run a number of simulations in *modified* environments, where I examine how the RL households adapt to changes in various model parameters.

6.2 Measures

To measure household welfare, I compute lifetime utility. Accuracy of interior consumption choices is evaluated using Euler errors. Additionally, I report the share of liquidity-constrained households. I illustrate policy functions, cash-on-hand distributions, and average life-cycle profiles for cash-on-hand, consumption, and assets with $\pm$ 1 standard deviation.

The Euler error is just the error in the household's first-order condition following from its problem. I compute *relative* Euler errors as Druedahl (2021). The relative Euler error averaged across time T and total number of households N_{HH} is

$$\bar{\mathcal{E}} = \frac{\sum_{i=0}^{N_{HH}-1} \sum_{t=0}^{T-1} \mathcal{E}_{it} \mathbf{1}(\text{uncon}_{it})}{\sum_{i=0}^{N_{HH}-1} \sum_{t=0}^{T-1} \mathbf{1}(\text{uncon}_{it})},$$

where

$$\mathcal{E}_{it} = \log_{10}\left(\left|\frac{\Delta_{it}}{c_{it}}\right|\right), \quad \Delta_{it} = c_{it}^{-\rho} - \beta R \, \mathbf{E}_t(c_{i,t+1}^{-\rho}),$$

and

$$\mathbf{1}(\text{uncon}_{it}) = \mathbf{1}\left(\tilde{c}_{it}^{-\rho} - \beta R \, \mathbf{E}_t(\tilde{c}_{i,t+1}^{-\rho}) \leqslant 0\right), \quad \tilde{c}_{it} = m_{it}. \tag{14}$$

Since the Euler equation can only be satisfied for unconstrained households, it is only calculated for unconstrained households, hence the dummy. The Euler error is therefore is a measure of the accuracy of the policy function, where the liquidity constraint does not bind. (14) states that a household is unconstrained if it hypothetically could have satisfied the Euler equation given its cash on hand, m_{it}. Using $a_{it} > 0$ is not a sufficient condition for the RL household, as optimal consumption choices are not guaranteed. I.e. simultaneous positive savings and a positive Euler error could be due to an inferior policy, where the RL household saves despite being constrained.

To illustrate the buffer-stock *target*, I will plot the consumption level that implies unchanged m_t:

$$\mathbf{E}_t(\Delta\, m_{t+1}) = \mathbf{E}_t(\, m_{t+1} - m_t) = 0 \iff \mathbf{E}_t(\tilde{\xi}_{t+1}) + ra_t = c_t \iff \frac{1 + m_t \cdot r}{1 + r} = c_t$$

Intuitively, this level corresponds to expected future income plus interest income and reflects the optimal trade-off between the need for self-insurance and the desire to consume today. All income shocks are drawn from the discretized log-normal distributions defined in Section 4, i.e. $\mathcal{G}_\xi$ and $\mathcal{G}_\psi$.

6.3 Main Results

Table 1 reports summary statistics from the all test simulations. Here, I briefly state the overall pattern.

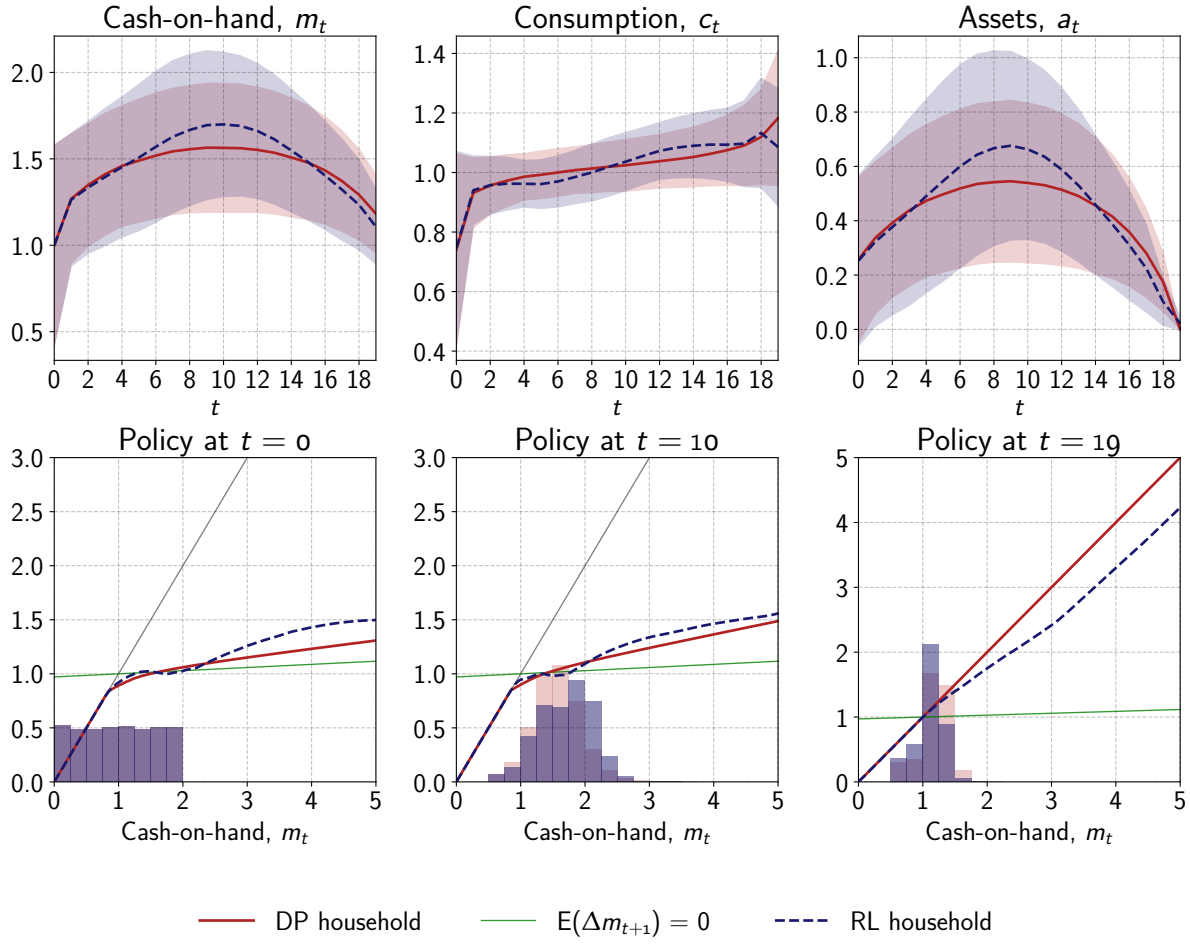
In all simulations, the RL household's behavior seem reasonable from an economic perspective. Their policy functions do not completely overlap with the optimal policy functions, but the overall shape is the same: They consume hand-to-mouth when liquidity constrained, and they save precautionarily for higher levels of wealth. Their life-cycle profiles also tend to follow the same overall pattern as those of the DP household: Assets and cash-on-hand are hump-shaped, and consumption gradually increases. Their average lifetime utility is consistently lower than that of DP households, but tends to be close.

However, RL household do evidently deviate from *perfectly* optimal behavior of DP households. What is driving the deviation? RL households are more frequently liquidity constrained, suggesting that some RL households under-save. However, there is no under-saving *on average*: The RL households generally maintain average levels of cash-on-hand that are similar to DP households. But the 5th percentile consistently under-save, and the 95th percentile consistently over-saves. This aligns with the fact that consumption on average is very close to the DP benchmark, but for the 5th percentile it is somewhat lower.

Generally, the tails of the distributions for RL households seem to be somewhat different compared to DP households. On one hand, RL households sometimes make "disastrous" saving decisions: the 95th percentile of Euler errors is almost consistently above -1, which is worse than hand-to-mouth households who average -1.[7] On the other hand, RL households sometimes make

[7] To put these values into perspective, an error of -1 corresponds to an average deviation of 1 percent, while -2 corresponds to 0.01 percent.

Fig 1. Baseline: Life-cycle profiles and policies.



very accurate decisions: the 5th percentile of their Euler errors is consistently lower than that of DP households.

A final note is that RL households tend end their life with a slightly positive level of assets (see Table 1), despite having no incentive to do so (e.g. no bequest motive). The precautionary savings motive from other periods seem to persist in the last period, which is also evident from policy functions for $t = 19$.

6.4 Baseline Simulations

The baseline environment is specified in Section 5.4. Figure 1 presents life-cycle profiles, policies and cash-on-hand distributions.

Optimal behavior. The DP policy function is concave, reflecting precautionary savings. For low levels of cash-on-hand, the DP household is liquidity constrained and consumes hand-to-

mouth. As it becomes wealthier, marginal propensity to consume starts to decline. At a certain threshold, the household reaches its *buffer-stock target*: for any m_t above this level, it dis-saves, as the consumption function lies above the line $\mathbf{E}(\Delta m_{t+1}) = 0$.

The life-cycle profiles for cash-on-hand and assets are hump-shaped. Households aim smooth consumption over time while also building a buffer stock. Early in its life, the household is below the buffer-stock target, so it saves to reach it. As time passes by, the precautionary savings motive diminishes, because there is less time to use it for shock-mitigation, and eventually the household begins to dis-save. These dynamics cause the consumption profile to increase in time.

RL households. The RL households approximate optimal behavior well. Policy functions and life-cycle profiles largely overlap for RL and DP households. The average lifetime utility for RL households is -0.38, compared to -0.36 for DP households. For comparison, hand-to-mouth households have an average utility of -0.65. The average relative Euler errors are -1.81 and -2.05 for RL and DP households, respectively.

Notably, the RL policy function starts to diverge from the optimal policy function around $m_t = 2.0$. This is because the RL household during its training primarily visits the regions of the state space where the optimal policy is located. I.e., the household rarely accumulates enough savings for cash-on-hand to exceed $m_t = 2.0$, since it learns relatively fast that saving beyond this threshold is suboptimal. This is also clearly reflected by the cash-on-hand distribution in the policy diagrams.

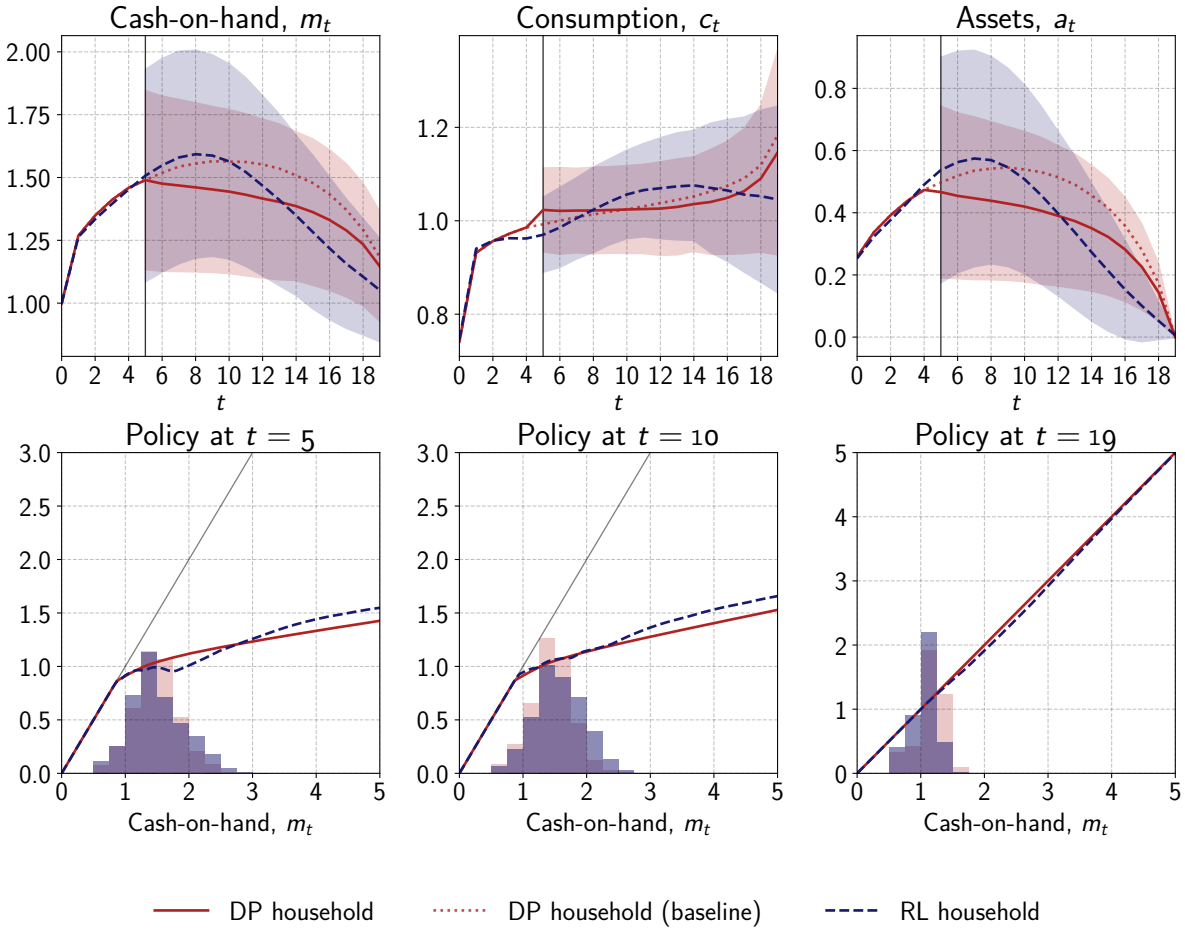
6.5 Parameter Experiments

6.5.1 Specifications

To examine how well RL households adapt to changes the environment, I do a series of parameter experiments. In these experiments, the households face a variation of the baseline environment, where a selected parameter value from the Buffer-Stock model is been altered. I call this the *modified* environment.

When the RL household is trained in the modified environment, I initialize it with the policy and value networks it reached during its baseline training. The idea behind this is to mimic a scenario in which the RL household initially finds itself in the baseline environment, and then the environment changes unexpectedly, and the household must adapt its old policy. The question

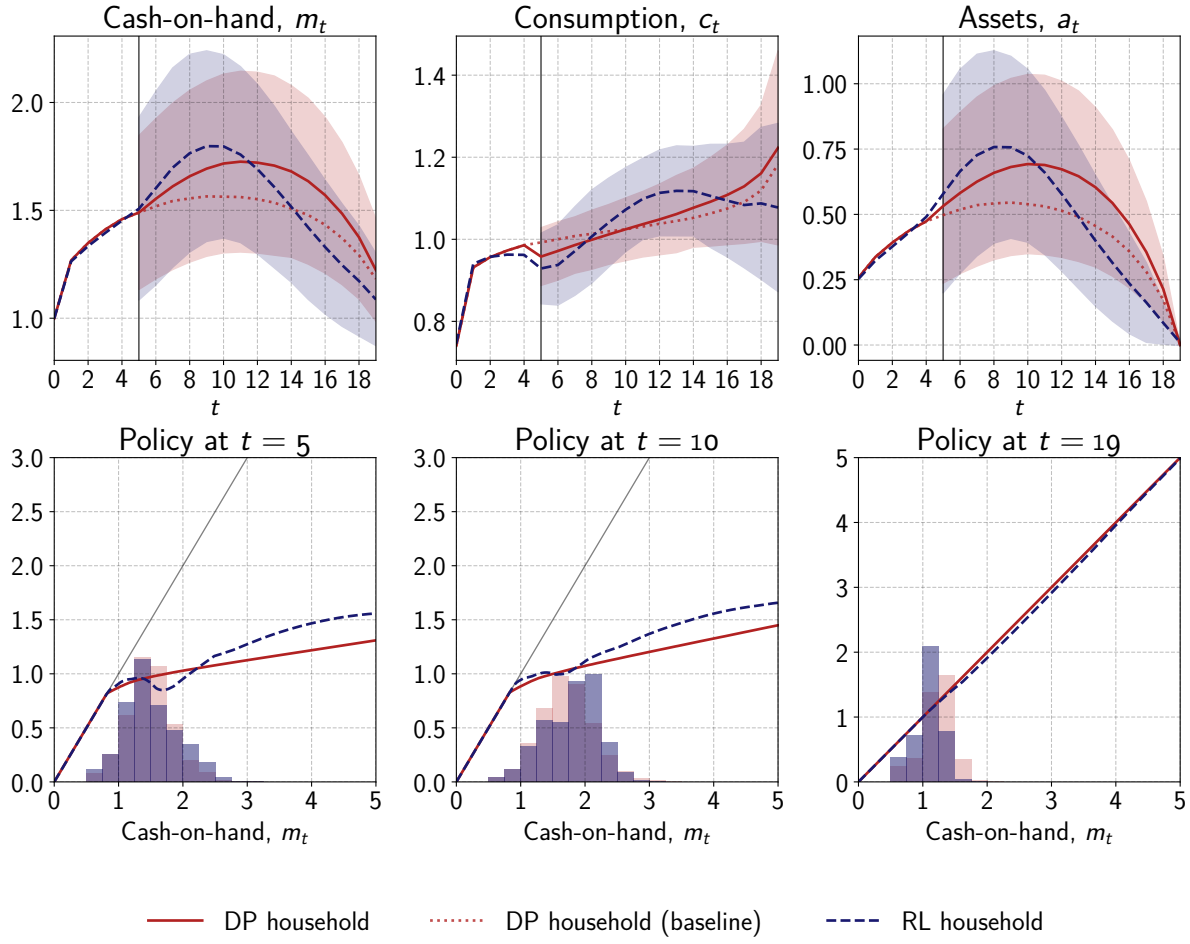
Fig 2. Decrease in the interest rate for $t \geqslant 5$. $R = 1.03 \rightarrow R = 1.02$. Life-cycle profiles and policies.



is whether the RL household can unlearn its old policy and learn the new optimal policy - or if its past experience traps it in a suboptimal behavior. The idea is that the value and policy functions reflect the RL household's current understanding of the environment. When the environment changes, the RL household is *biased* by its current understanding, and this may affect how the household adapts to the modified environment.

I assume that the household faces the modified environment for $t \geqslant 5$. The simulations follow the same procedure described in Section 6.1, except that $t = 5$ is the start period. Initial cash-on-hand for household i is set to the cash-on-hand level it reached in the baseline simulation at $t = 5$. I emphasize that in the parameter experiments the RL household *does not* learn and adapt to the modified environment in *real time*, becuase they re-live periods $t = [5,19]$ repeatedly to learn a new policy for the modified environment.

Fig 3. Increase in the interest rate for $t \geqslant 5$. $R = 1.03 \rightarrow R = 1.04$. Life-cycle profiles and policies.



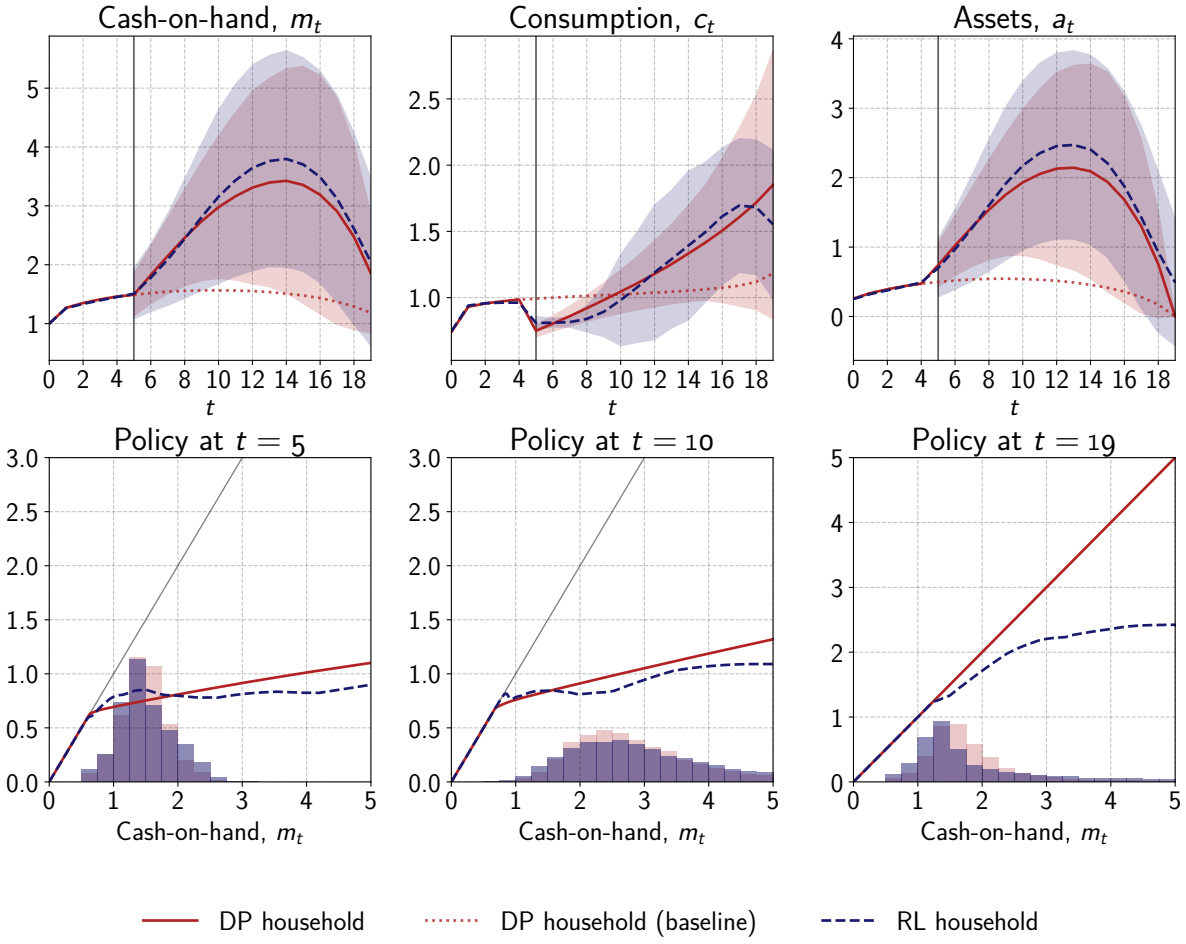
6.5.2 Change in Interest Rate

Figure 2 shows a *decrease* in R, and Figure 3 shows an *increase* in R. The dotted line in the figures show the baseline life-cycle profiles, i.e. the DP profile from Figure 1.

Optimal behavior. A lower interest rate reduces the relative payoff to future consumption, making the DP households to shift consumption forward. While precautionary savings still applies, it is weighed against the diminished return on assets. Therefore, the DP households lower its buffer-stock target, since holding large precautionary savings becomes less valuable under the lower interest rate. The opposite holds for a higher interest rate.

RL households. The RL households respond differently to a decrease in the interest rate compared to DP households. First, their response is lagged: instead of adjusting savings immediately, the RL households remain on the baseline path for a few periods before reducing their savings. Second, once the RL households do adjust, the reduction in their savings is excessive compared

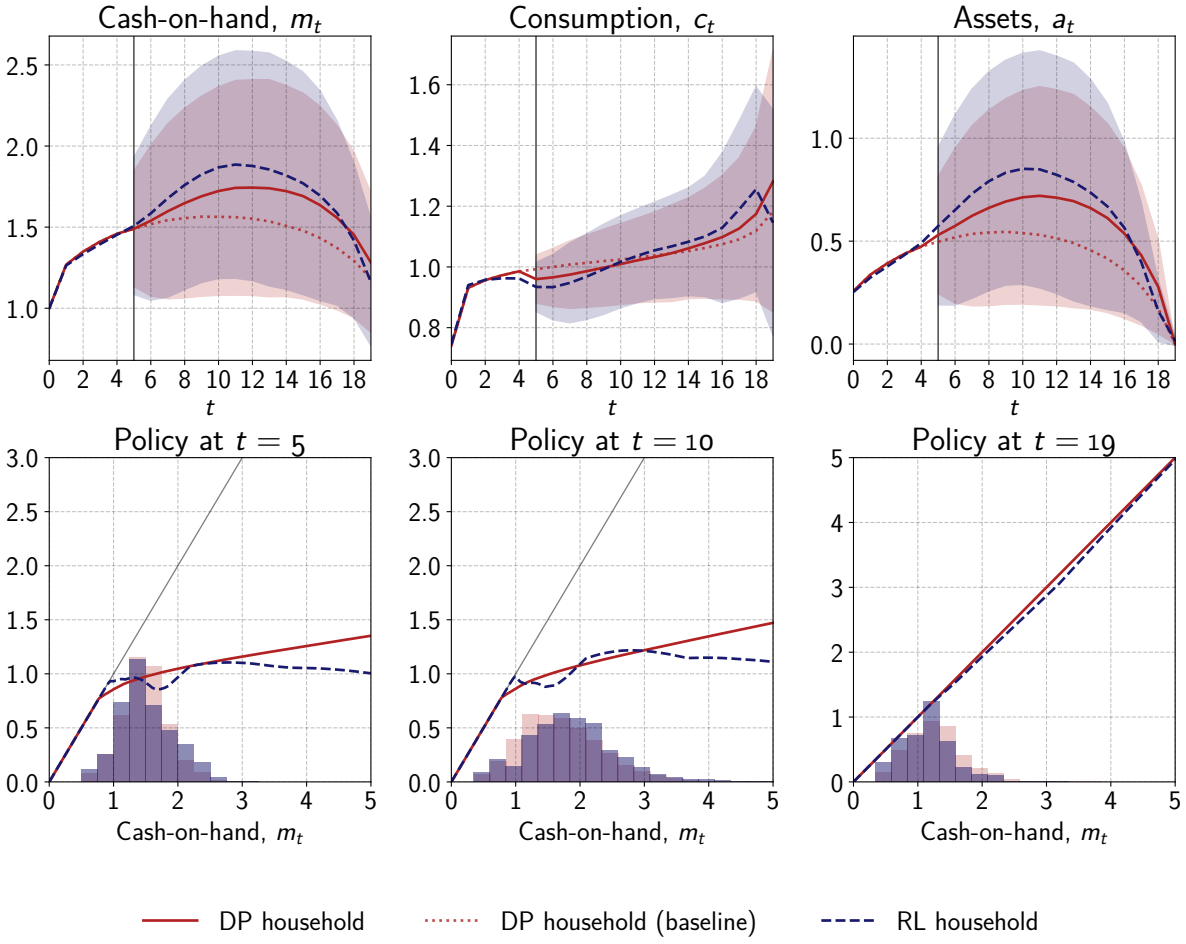
Fig 4. Increase in permanent income risk for $t \geqslant 5$. $\sigma_\psi = 0.1 \rightarrow \sigma_\psi = 0.3$. Life-cycle profiles and policies.



to the DP benchmark. It is unclear, why the RL households respond as they do. Are they biased by their previous experience? What is the economic reasoning? If households learned in real time, lagged responses would seem reasonable, because the RL household would need time to adjust to the new environment. But in this setup, the RL household lives through the remaining portion of its life repeated times, so a lagged response seems harder to justify.

The response to an increase in the interest rate is strikingly similar in terms of the shape of life-cycle profiles: an initial over-saving followed by excessive consumption. Although it seems reasonable to initially over-save in the case of an *increased* interest rate, it seems counterintuitive to over-save for a *decreased* interest rate. This RL behavior is hard to understand from an economic perspective.

Fig 5. Increase in transitory income risk for $t \geqslant 5$. $\sigma_\xi = 0.1 \rightarrow \sigma_\xi = 0.3$. Life-cycle profiles and policies.



6.5.3 Increased Income Risk

Figures 4 and 5 show the effect of increased variance in permanent and transitory income, respectively. I also document the effects of a decrease in μ and an increase in π, but since these changes correspond to an increase in the variance of transitory income, I leave these figures in the Appendix.

Optimal behavior. Optimal response is to increase precautionary savings, when income risk increases. The increase is more pronounced in response to greater *permanent* income risk, contrary to *transitory* income risk. This is because permanent income is the persistent income component. I.e., low-income (high-income) shocks are more likely to be followed by subsequent low-income (high-income) shocks. Exactly because of this, the share of constrained households decreases - it becomes more costly to be constrained - and cash-on-hand distributions that become more dispersed, and wealth inequality rises.

RL households. RL households respond to raised income risk by increasing their precautionary savings - like DP households - although there is slight over-saving on average.

An interesting observation is that RL households appear so concerned about low-income scenarios that they end their life cycle with substantial positive wealth (in the case of increased permanent income risk). Another odd observation is that the RL households *reduce* their consumption during the final two periods. This reduced consumption in the final periods leads to a significant drop in lifetime utility for the RL households. The DP households average a lifetime utility of 0.19, while the RL households average only 0.03. The RL households leave an average of 0.49 of unused wealth in the final period, which corresponds to a loss of approximately 0.14 in lifetime utility.[8] Therefore, the difference in average lifetime utility between DP and RL households can largely be explained by RL households' unused wealth in the last period.

In fact, a drop in consumption in the final periods is a common theme across all simulations if one takes a closer look, although the drop is most pronounced in the case of increased permanent income risk. This seems counterintuitive and difficult to explain, especially given that the RL household tends to leave a positive level of assets behind.

[8] If the unused wealth were consumed in the last period, the household's total consumption would amount to approximately $0.49 + 1.5 \approx 2.0$. With log utility, this results in a utility difference of $0.96^{18}(\log(2) - \log(1.5)) \approx 0.14$.

Table 1. Life-cycle Simulations

	Baseline			$\sigma_\xi = 0.3$		$\sigma_\psi = 0.3$		R= 1.04		$R = 1.02$		$\pi = 0.5$		$\mu = 0.25$	
	RL	DP	HTM	DP	RL	DP	RL	DP	RL	DP	RL	DP	RL	DP	RL
Consumption															
Average	1.02	1.02	1.00	1.02	1.02	1.15	1.14	1.02	1.02	1.01	1.01	1.03	1.02	1.02	1.02
5th percentile	0.76	0.79	0.50	0.71	0.70	0.74	0.79	0.83	0.74	0.74	0.68	0.57	0.50	0.72	0.68
95th percentile	1.23	1.22	1.28	1.36	1.35	1.96	1.98	1.26	1.23	1.19	1.19	1.53	1.51	1.32	1.28
Standard deviation	0.16	0.15	0.23	0.22	0.23	0.51	0.48	0.16	0.17	0.15	0.16	0.29	0.29	0.19	0.20
Assets															
Average	0.45	0.41	0.00	0.52	0.57	1.21	1.36	0.49	0.45	0.35	0.36	0.69	0.65	0.54	0.54
5th percentile	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
95th percentile	1.07	0.94	0.00	1.35	1.42	3.27	4.13	1.10	1.11	0.83	1.02	1.88	1.81	1.15	1.21
Standard deviation	0.35	0.29	0.00	0.44	0.50	1.19	1.28	0.34	0.37	0.26	0.33	0.61	0.61	0.36	0.41
Average last period	0.02	0.00	0.00	0.00	0.01	0.00	0.49	0.00	0.01	0.00	0.01	0.00	0.03	0.00	0.02
Cash-on-hand															
Average	1.46	1.43	1.00	1.54	1.59	2.36	2.49	1.52	1.47	1.36	1.37	1.71	1.68	1.56	1.56
5th percentile	0.76	0.79	0.50	0.71	0.70	0.97	0.91	0.83	0.74	0.74	0.68	0.57	0.50	0.74	0.68
95th percentile	2.20	2.05	1.28	2.58	2.70	4.92	5.73	2.22	2.25	1.94	2.11	3.19	3.14	2.30	2.37
Standard deviation	0.44	0.39	0.23	0.58	0.63	1.50	1.60	0.43	0.46	0.36	0.42	0.79	0.79	0.47	0.52
Life-time utility															
Average	-0.38	-0.36	-0.64	-0.47	-0.51	0.19	0.03	-0.33	-0.36	-0.40	-0.42	-0.64	-0.70	-0.45	-0.50
5th percentile	-2.69	-2.67	-2.93	-3.03	-3.08	-2.57	-2.67	-2.70	-2.75	-2.78	-2.80	-3.63	-3.72	-3.03	-3.15
95th percentile	1.26	1.28	1.00	1.55	1.51	2.97	2.78	1.32	1.29	1.20	1.19	1.83	1.79	1.35	1.32
Standard deviation	1.25	1.25	1.24	1.42	1.42	1.68	1.62	1.26	1.26	1.25	1.25	1.68	1.70	1.37	1.39
Relative Euler errors															
Average	-1.80	-2.05	-1.08	-2.06	-1.46	-1.39	-1.37	-2.05	-1.66	-2.05	-1.72	-2.10	-1.49	-2.08	-1.59
5th percentile	-2.80	-2.11	-1.49	-2.16	-2.32	-2.05	-2.28	-2.12	-2.55	-2.11	-2.54	-2.25	-2.54	-2.16	-2.71
95th percentile	-1.07	-2.00	-0.59	-2.00	-0.78	-1.08	-0.76	-2.00	-1.00	-1.99	-1.12	-1.99	-0.59	-2.01	-0.74
Standard deviation	0.52	0.03	0.37	0.05	0.47	0.37	0.51	0.04	0.48	0.04	0.45	0.09	0.60	0.05	0.65
Share of constrained HHs															
Average	0.12	0.11	0.24	0.12	0.12	0.08	0.09	0.10	0.12	0.12	0.14	0.13	0.15	0.10	0.11

7 Discussion

7.1 Real-Time Learning

The RL household learns by simulating itself in multiple life cycles. I.e., the household lives through thousands of life cycles to learn a policy, so when its "real life" begins it can execute its best policy from the outset. In other words, there is a clear separation of the *training* and the *test* phases. In reality, this is obviously not the case. Real households do not get to "replay" their life cycle - they experience a single lifetime, and learning happens in real time as life unfolds.

In theory, it is possible to have the RL household learn in *real time* rather than through learning over repeated life cycles. To achieve this, the RL household could placed in an infinite-horizon environment of the Buffer-Stock model. When the environment changes, the household must adjust its policy in parallel to the environment change, as opposed to being pre-trained in a separate simulation. This approach is appealing because here learning occurs simultaneously with the environmental change, as it does in reality.

However, it is not straightforward how to balance exploration and exploitation when there is no distinct training phase. In an isolated training phase, the household experiments to find the best policy (exploration), and in an isolated testing phase the household uses the policy it converged to during training (exploitation). But with learning in real time, training and testing occur simultaneously, so how should exploration and exploitation be weighted? An intuitive way to frame this is that in an isolated training phase, the household can afford to experiment freely because there are no real consequences - it learns from mistakes without affecting its lifetime utility. However, with real-time learning, every decision has real consequences, and if the household experiments with a poor policy for too long, it could harm its lifetime utility.

7.2 Economic Interpretation

The life-cycle simulations showed that the RL household's behavior mostly seems reasonable. However, it does evidently deviate from perfectly optimal behavior - especially in the tails of the distribution, where consumption is significantly lower and Euler errors are greater compared to DP households. What factors are driving the deviations?

The mechanics of the learning process are well-defined; it follows an algorithm, which also has an intuitive appeal. The RL household attempts to predict the value of its consumption-saving strategy, and adjusts it over time to maximize its lifetime utility. In this sense, the RL household is *forward-looking* and a *rational* agent, although it has a limited *capacity*.

But its economic interpretation remains unclear: What drives the RL household towards a certain policy? Clearly, the it updates its policy based on feedback from the environment, but if it converges to a suboptimal policy, why does this happen? There is no clear economic explanation. One could attribute this to technical factors such as poor hyperparameter choices, but these are *engineering* explanations rather than *economic* ones.

An example of this is when the interest rate decreases (see Figure 2). The optimal response is to increase current consumption and reduce savings. Yet, the RL households have a lacked response, where it initially over-saves and later under-saves. While this is not implausible behavior, the RL framework does not provide any economic explanation. The interpretation of the RL policy function itself is also challenging. In general, it tends to lie close to the optimal policy function, but still deviates somewhat by fluctuating around with no obvious pattern. What should be made of this? Unlike systematic deviations that could be explained by known behavioral biases, these fluctuations appear arbitrary.

7.3 Empirical Connection

An economic theory must have capacity to explain empirical observations. This begs the question if an RL household accurately reflects the behavior of a real household?

A motivation for modeling households as reinforcement learners is that RL agents are rational, but limited in their information of the environment and capacity to solve their problem perfectly. I.e., the RL agent may be interpreted as being boundedly rational. This may align with empirical evidence. For example, habit models suggest that past consumption behavior influences future consumption (see e.g. Carroll et al., 2000; Furman and Seamans, 2018; Ganong and Noel, 2019), and models of sticky expectations and information rigidities argue that agents update their expectations slowly and imperfectly (e.g., Coibion and Gorodnichenko, 2015 and Carroll et al., 2021). Could this kind of behavior be captured in by modeling a household as a reinforcement learner? In theory maybe, but it would require a real-time learning setup as discussed earlier, and the challenge of interpreting the RL household's behavior remains.

Could hyperparameters be used to model a kind of heterogeneity in learning among households? Ameriks et al. (2003) show that individuals who report actively planning their finances tend to have higher net worth than those who do not, after controlling for income. This may resemble RL households with a low degree of exploration that fail to discover the optimal policy. In the Appendix, I document how changes in hyperparameters affect the household's policy, and it is clear that hyperparameter settings like the degree of exploration have an impact. However, it is not predictable in what way the policy is changed - after all, exploration is just noise in the household's consumption choices during training. Holland and Miller (1991) emphasized that modeling boundedly rational agents is inherently challenging because "there is only one way to be fully rational, but there are many ways to be less rational". This is also the challenge of the RL household.

7.4 Best as a Computational Tool?

Although an RL agent may resemble how we think of economic agents - being rational, forward-looking, yet biased by experience and limited in capacity - the learning process and outcome is opaque, and the RL household's economic reasoning remains unclear. This makes it difficult to take the RL household's suboptimal behavior seriously. Policy evaluation in a model with RL households that potentially act suboptimally may lack credibility, because RL behavior is hard to interpret. A good understanding of how to "shape" both the learning process and the resulting policy would be beneficial.

In addition, reinforcement learning is a rather complex way of modeling economic agents. Implementation involves extensive trial-and-error tuning and many decisions must be made about algorithm type, exploration, learning rates, functional approximation, and many of these only have a vague economic interpretation. This lack of interpretability is problematic if reinforcement learning is meant to serve as a behavioral model of economic agents, but less problematic if pure performance is the goal - which is arguably what reinforcement learning algorithms are designed for. Given this, reinforcement learning may be best suited as a computational tool for solving large economic models rather than as a behavioral model of economic agents.

8 Conclusion

This thesis studied reinforcement learning (RL) households in the Buffer-Stock model. Rather than solving its optimization problem perfectly, the RL household must learn how to behave through experience in the model environment. This may be in line with how we think of real economic agents. Using the Deep Deterministic Policy Gradient algorithm, I trained an RL household to learn a consumption-saving policy in a finite-horizon setting and compared its behavior to the optimal dynamic programming solution. The RL household tends to act rationally and achieves near-optimal behavior in some cases. However, in other cases, it deviates more distinctly from optimal behavior. These deviations might reflect the RL household's bounded rationality, but its behavior remains opaque and challenging to interpret in economic terms. Models that rely on sub-optimal RL households may risk losing credibility, if their predictions are difficult to interpret from an economic perspective.

References

J. Ameriks, A. Caplin, and J. Leahy. Wealth Accumulation and the Propensity to Plan*. *The Quarterly Journal of Economics*, 118(3):1007–1047, Aug. 2003. ISSN 0033-5533. doi: 10.1162/00335530360698487. URL https://doi.org/10.1162/00335530360698487.

W. B. Arthur. Designing economic agents that act like human agents: A behavioral approach to bounded rationality. *American Economic Review*, 81(2):353–59, 1991. URL https://EconPapers.repec.org/RePEc:aea:aecrev:v:81:y:1991:i:2:p:353-59.

M. Azinovic, L. Gaegauf, and S. Scheidegger. Deep Equilibrium Nets. *International Economic Review*, 63(4):1471–1525, 2022. ISSN 1468-2354. doi: 10.1111/iere.12575. URL https://onlinelibrary.wiley.com/doi/abs/10.1111/iere.12575. _eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/iere.12575.

R. Bellman. *Dynamic Programming*. Dover Publications, 1957. ISBN 9780486428093.

C. D. Carroll. Buffer-Stock Saving and the Life Cycle/Permanent Income Hypothesis. *The Quarterly Journal of Economics*, 112(1):1–55, 1997. ISSN 0033-5533. URL https://www.jstor.org/stable/2951275. Publisher: Oxford University Press.

C. D. Carroll. The method of endogenous gridpoints for solving dynamic stochastic optimization problems. *Economics Letters*, 91(3):312–320, 2006. ISSN 0165-1765. doi: https://doi.org/10.1016/j.econlet.2005.09.013. URL https://www.sciencedirect.com/science/article/pii/S0165176505003368.

C. D. Carroll, J. Overland, and D. N. Weil. Saving and Growth with Habit Formation. *The American Economic Review*, 90(3):341–355, 2000. ISSN 0002-8282. URL https://www.jstor.org/stable/117332. Publisher: American Economic Association.

C. D. Carroll, M. B. Holm, and M. S. Kimball. Liquidity constraints and precautionary saving. *Journal of Economic Theory*, 195:105276, July 2021. ISSN 0022-0531. doi: 10.1016/j.jet.2021.105276. URL https://www.sciencedirect.com/science/article/pii/S0022053121000934.

O. Coibion and Y. Gorodnichenko. Information Rigidity and the Expectations Formation Process: A Simple Framework and New Facts. *American Economic Review*, 105(8):2644–2678, Aug. 2015. ISSN 0002-8282. doi: 10.1257/aer.20110306. URL https://www.aeaweb.org/articles?id=10.1257/aer.20110306.

A. Deaton. Saving and liquidity constraints. *Econometrica*, 59(5):1221–1248, 1991. ISSN 00129682, 14680262. URL http://www.jstor.org/stable/2938366.

C. Deissenberg, S. van der Hoog, and H. Dawid. EURACE: A massively parallel agent-based model of the European economy. *Applied Mathematics and Computation*, 204(2):541–552, Oct. 2008. ISSN 0096-3003. doi: 10.1016/j.amc.2008.05.116. URL https://www.sciencedirect.com/science/article/pii/S0096300308003019.

J. Druedahl. A Guide on Solving Non-convex Consumption-Saving Models. *Computational Economics*, 58(3):747–775, Oct. 2021. ISSN 1572-9974. doi: 10.1007/s10614-020-10045-x. URL https://doi.org/10.1007/s10614-020-10045-x.

J. Fernández-Villaverde, G. Nuno, and J. Perla. Taming the Curse of Dimensionality:Quantitative Economics with Deep Learning. PIER Working Paper Archive 24-034, Penn Institute for Economic Research, Department of Economics, University of Pennsylvania, Oct. 2024. URL https://ideas.repec.org/p/pen/papers/24-034.html.

J. Furman and R. Seamans. AI and the Economy, May 2018. URL https://papers.ssrn.com/abstract=3186591.

P. Ganong and P. Noel. Consumer Spending during Unemployment: Positive and Normative Implications. *American Economic Review*, 109(7):2383–2424, July 2019. ISSN 0002-8282. doi: 10.1257/aer.20170537. URL https://www.aeaweb.org/articles?id=10.1257/aer.20170537.

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

E. Hall-Hoffarth. Non-linear approximations of DSGE models with neural-networks and hard-constraints, Oct. 2023. URL http://arxiv.org/abs/2310.13436. arXiv:2310.13436 [econ] version: 1.

J. Han, Y. Yang, and W. E. DeepHAM: A Global Solution Method for Heterogeneous Agent Models with Aggregate Shocks, Feb. 2022. URL http://arxiv.org/abs/2112.14377. arXiv:2112.14377 [cs, econ, q-fin].

E. Hill, M. Bardoscia, and A. Turrell. Solving Heterogeneous General Equilibrium Economic Models with Deep Reinforcement Learning, Mar. 2021. arXiv:2103.16977 [cs, econ, q-fin, stat].

J. H. Holland and J. H. Miller. Artificial Adaptive Agents in Economic Theory. *The American Economic Review*, 81(2):365–370, 1991. ISSN 0002-8282. URL https://www.jstor.org/stable/2006886. Publisher: American Economic Association.

K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, Jan. 1989. ISSN 0893-6080. doi: 10.1016/0893-6080(89)90020-8. URL https://www.sciencedirect.com/science/article/pii/0893608089900208.

K. L. Judd. *Numerical Methods in Economics*, volume 1 of *MIT Press Books*. The MIT Press, December 1998. ISBN ARRAY(0x99a5f130). URL https://ideas.repec.org/b/mtp/titles/0262100711.html.

H. Kase, L. Melosi, and M. Rottner. Estimating Nonlinear Heterogeneous Agent Models with Neural Networks. 2024.

T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning, July 2019. URL http://arxiv.org/abs/1509.02971. arXiv:1509.02971 [cs].

L. Maliar, S. Maliar, and P. Winant. Deep learning for solving dynamic economic models. *Journal of Monetary Economics*, 122:76–101, Sept. 2021. ISSN 0304-3932. doi: 10.1016/j.jmoneco.2021.07.004. URL https://www.sciencedirect.com/science/article/pii/S0304393221000799.

V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. A. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. URL https://api.semanticscholar.org/CorpusID:205242740.

J. Payne, A. Rebei, and Y. Yang. Deep Learning for Search and Matching Models. 2024.

R. A. Shi. *Deep reinforcement learning and macroeconomic modelling*. phd, University of Warwick, Apr. 2023. URL http://webcat.warwick.ac.uk/record=b3988261.

R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, second edition, 2018. URL http://incompleteideas.net/book/the-book-2nd.html.

9 Appendix

Fig 6. Learning process.

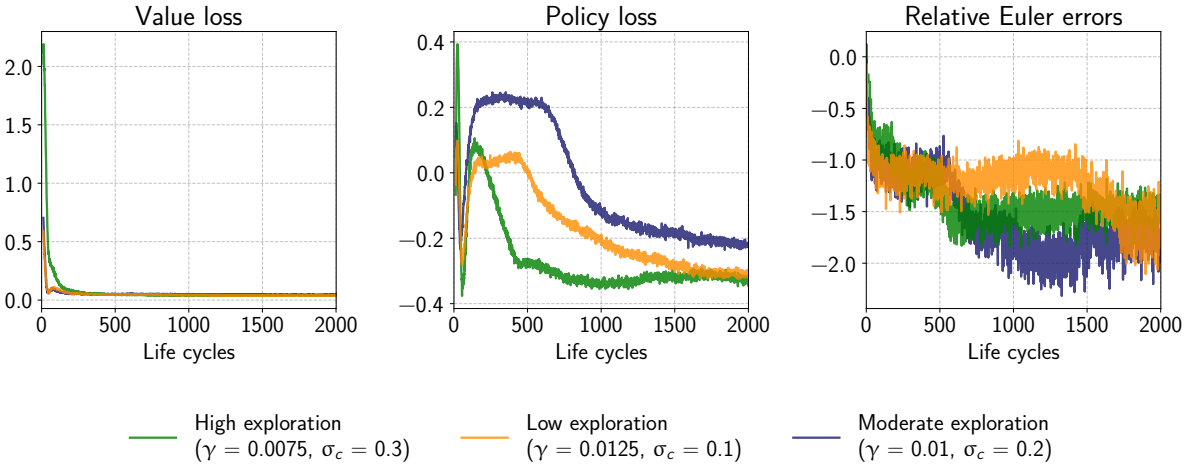


Fig 7. Learning process.

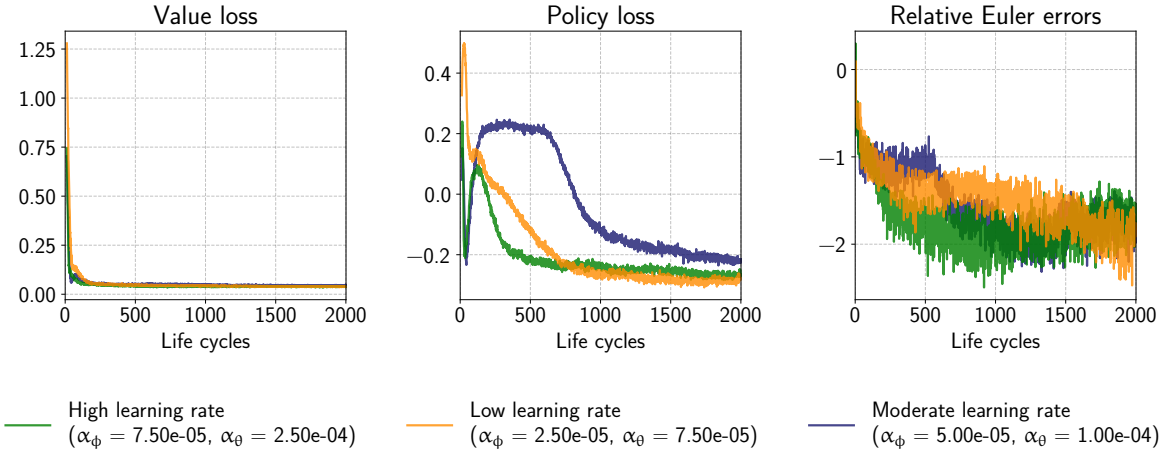


Fig 8. Variation in learning rates.

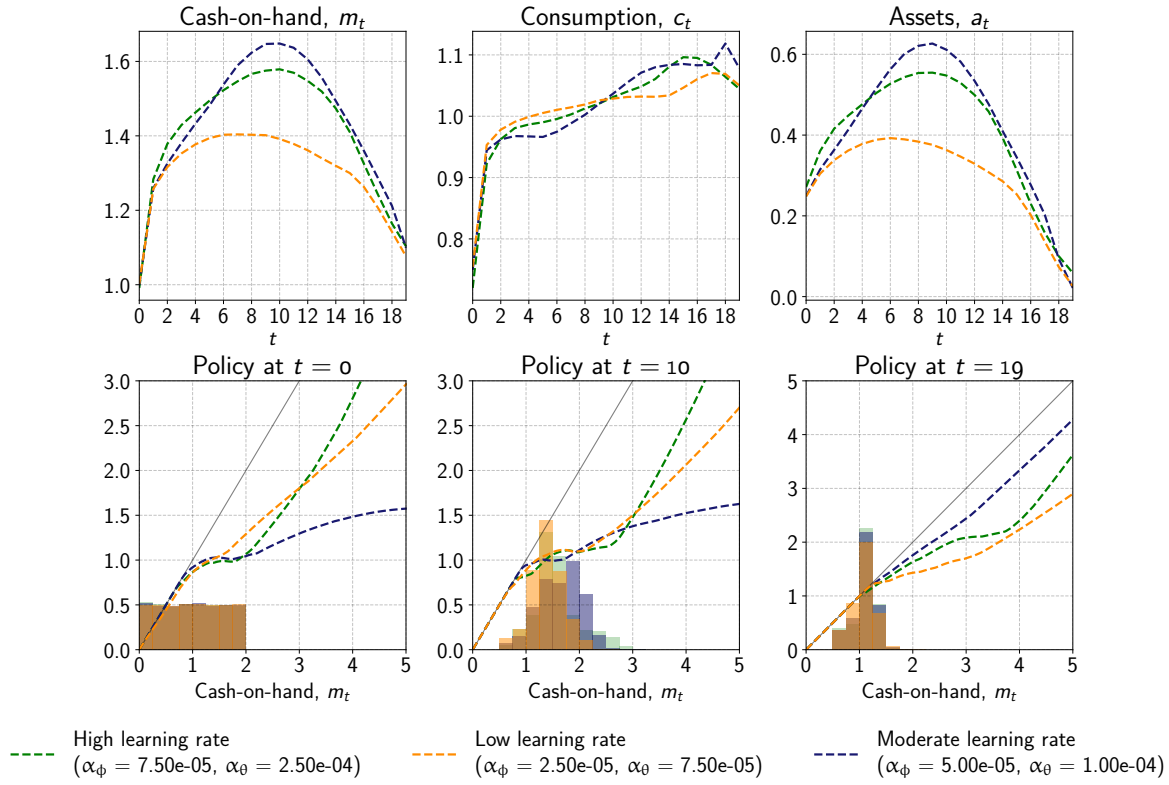


Fig 9. Variation in exploration. Life-cycle profiles and policies.

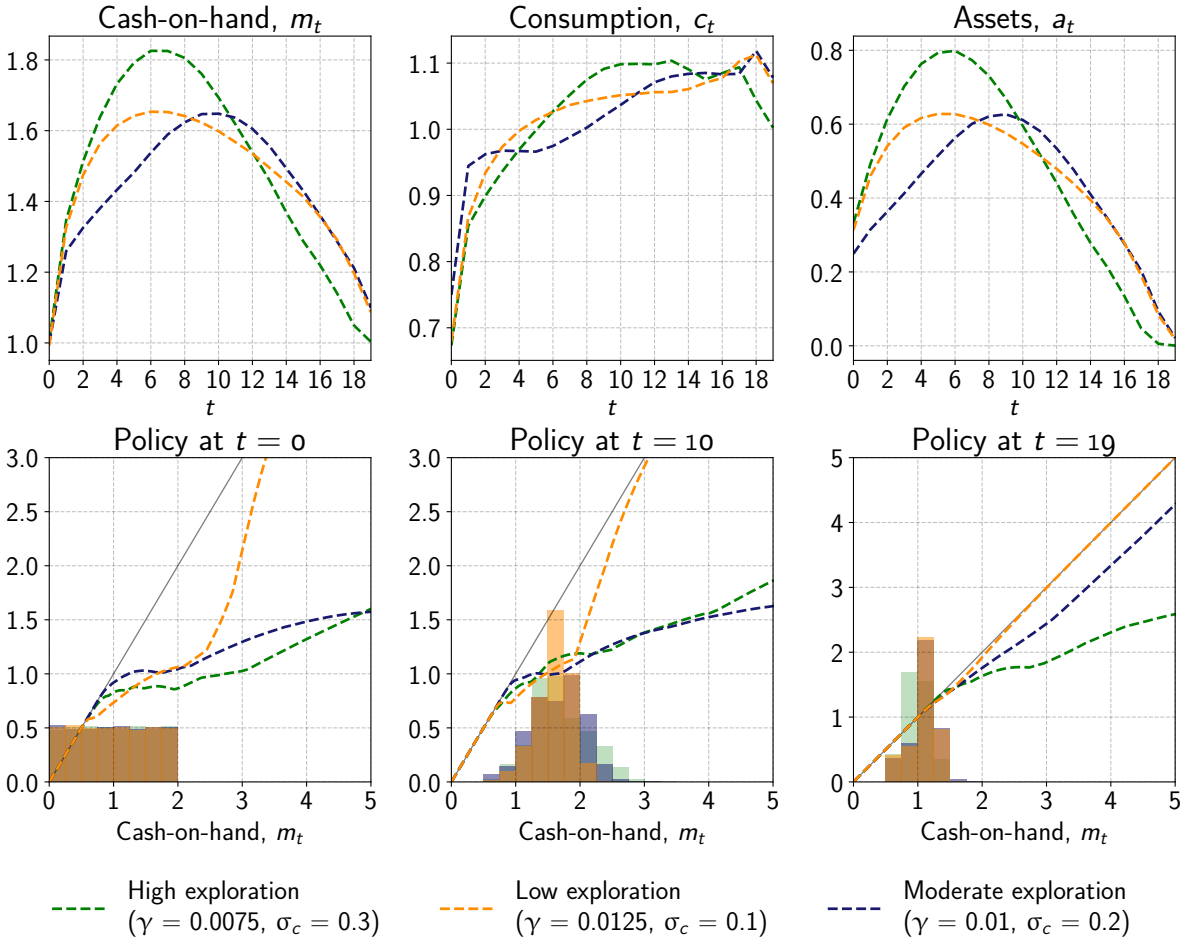


Fig 10. Increase probability of low-income shocks: $\pi = 0.1 \rightarrow \pi = 0.5$. Life-cycle profiles and policies.

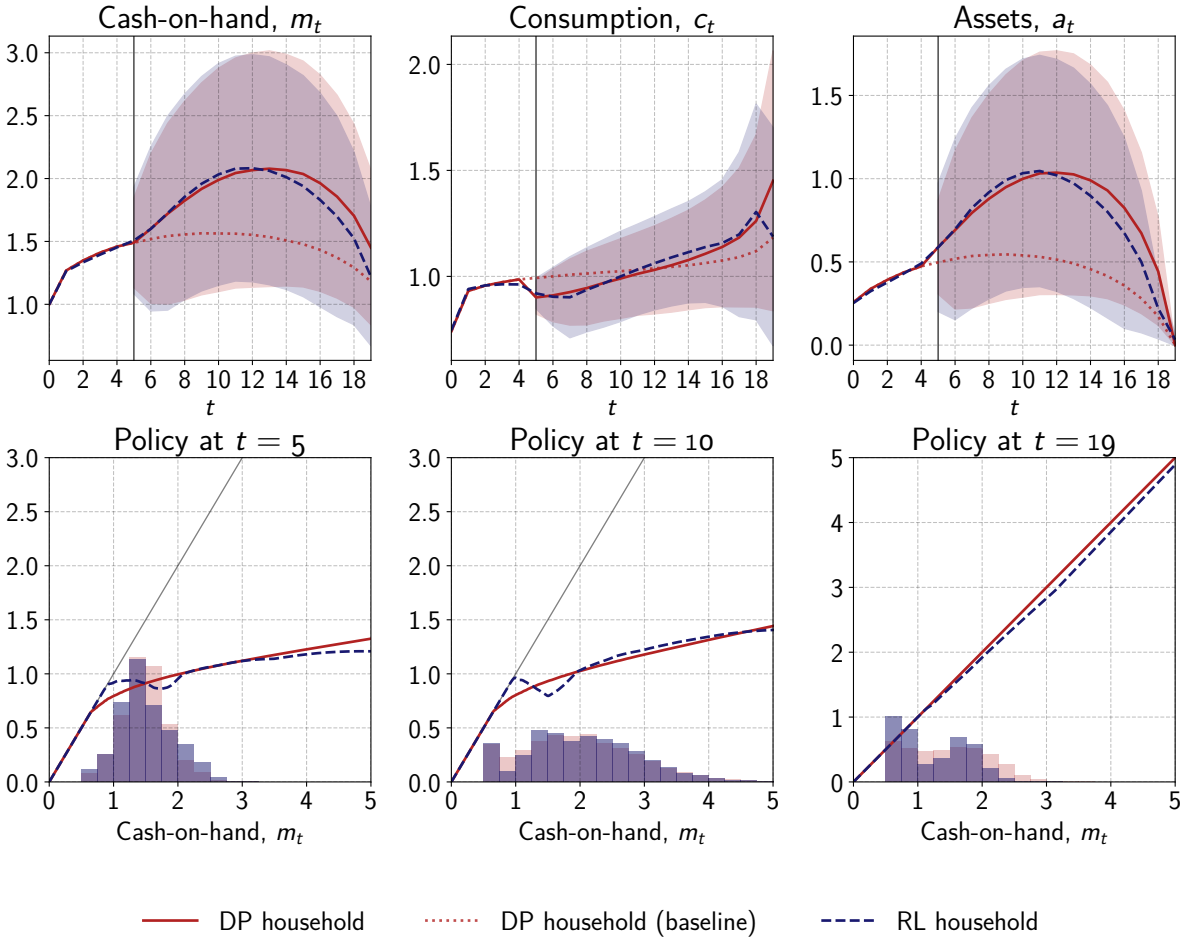


Fig 11. "Worse" low-income shocks: $\mu = 0.5 \rightarrow \mu = 0.25$. Life-cycle profiles and policies.